A

Financial Advisor AI Chatbot

Project documentation

Version: 13 October 2026

Stack: Flask · React · Google Gemini · Postgres · Docker

Contents: setup, architecture, backend, frontend, API, security, testing, deployment, troubleshooting, user guide

Screenshots and API examples were captured from the running application.

Contents

1 About This Documentation

2 Local Setup

3 Architecture

4 Backend

5 Frontend

6 API Reference

7 Security

8 Testing

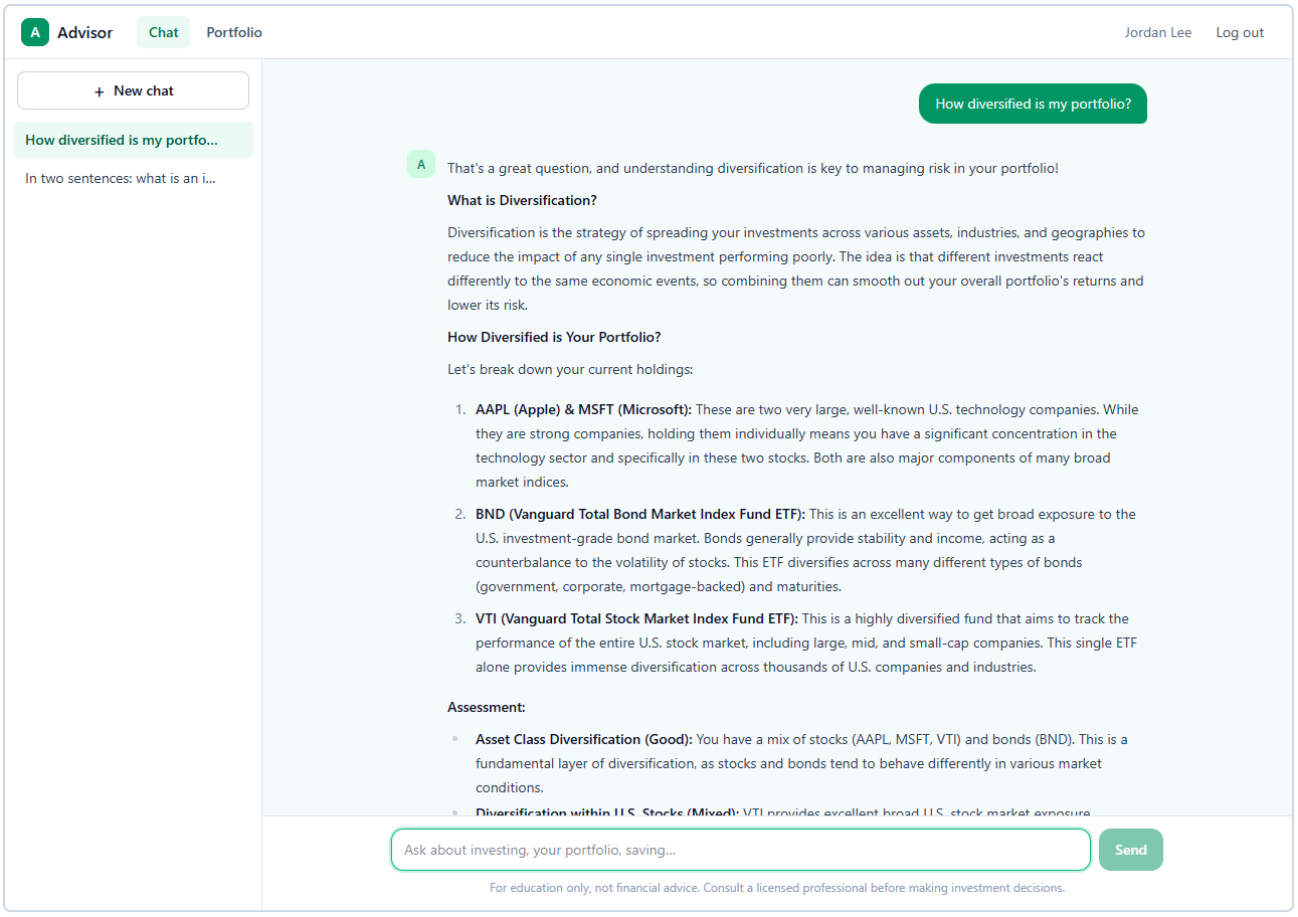
9 Deployment

10 Troubleshooting

11 User Guide

1 About This Documentation

Purpose: Index of the project documentation, and which documents to read for your role.



Chat page answering a question about the user's portfolio

Documents

Document	What it covers	Main readers
SETUP.md	Local development setup, Docker Compose, verifying it works	New developers
ARCHITECTURE.md	System diagram, database schema, request flows	Everyone technical
BACKEND.md	Flask app structure, models, routes, services, config	Backend developers
FRONTEND.md	React structure, routing, state, streaming, styling	Frontend developers
API.md	Every endpoint, with real request and response examples	Frontend devs, API consumers, QA
SECURITY.md	Auth, roles, rate limits, validation, audit logging	Security reviewers
TESTING.md	Running the test suites, what they cover, mocking	Developers, QA
DEPLOYMENT.md	Supabase + Render + Vercel, or one Docker host	DevOps
TROUBLESHOOTING.md	Common errors and how to fix them	Everyone
USER_GUIDE.md	How to use the app, with screenshots	End users, support

Reading order by role

- **Backend developer:** SETUP → BACKEND, then ARCHITECTURE, SECURITY and API as reference.
- **Frontend developer:** SETUP → FRONTEND, then API and ARCHITECTURE.
- **DevOps:** DEPLOYMENT, then ARCHITECTURE and SETUP.
- **Security reviewer:** SECURITY → ARCHITECTURE, then BACKEND (code patterns) and TESTING (coverage).
- **End user or support:** USER_GUIDE, then TROUBLESHOOTING.

Evidence

The screenshots in `screenshots/` were captured on 2026-10-13 from the app running locally (`python main.py` + `npm run dev`) against a throwaway database, with a real Gemini API key and live Yahoo Finance prices. The API examples in API.md are real responses from that same run. The test output images in TESTING.md are the actual output of `pytest -v` and `npm test` on the same day.

Keeping these docs accurate

- Update the relevant document in the same commit as the code change.
- Use real commands (`python main.py` , not "start the server") and real examples.
- When the UI changes visibly, retake the affected screenshots.
- Update the **Last Updated** line at the bottom of each document you change.

2 Local Setup

Purpose: Get the app running on your machine for development, from a fresh clone.

Overview

You run two processes: the Flask backend on port 5000 and the Vite frontend on port 5173, which forwards `/api` calls to the backend. Alternatively, Docker Compose runs the whole production-like stack on port 8080.

Prerequisites

Tool	Version used	Needed for
Python	3.13	Backend
Node.js	24	Frontend
Git	any	Cloning
Docker Desktop	optional	Running the full stack
Gemini API key	free	Chat replies. Get one at https://aistudio.google.com/apikey

Clone the repo

```
git clone https://github.com/<you>/financial-advisor-chatbot.git
cd financial-advisor-chatbot
```

Backend

From `backend/`:

```
python -m venv venv
venv\Scripts\Activate.ps1          # PowerShell
# source venv/Scripts/activate     # Git Bash on Windows
# source venv/bin/activate         # macOS / Linux
pip install -r requirements.txt
cp .env.example .env
```

Edit `backend/.env` and set at least:

```
GEMINI_API_KEY=your-key-here
```

The other values in `.env.example` are fine for development. In development `JWT_SECRET_KEY` and `SECRET_KEY` fall back to built-in dev values if left blank.

Start the server:

```
python main.py
```

`main.py` applies database migrations first, creating `backend/instance/dev.db` (SQLite) on the first run, then serves on http://127.0.0.1:5000. It reloads automatically when you change a `.py` file. If a reload hits a syntax error the server stops, so restart it after fixing the error.

To use a different database file, for example to keep demo data separate:

```
$env:DATABASE_URL = "sqlite:///demo.db"; python main.py
```

Frontend

In a second terminal, from `frontend/`:

```
npm install
npm run dev
```

Open http://localhost:5173. The dev server forwards `/api/*` to `http://127.0.0.1:5000`. To point it elsewhere, set `API_PROXY_TARGET`:

```
$env:API_PROXY_TARGET = "http://127.0.0.1:8000"; npm run dev
```

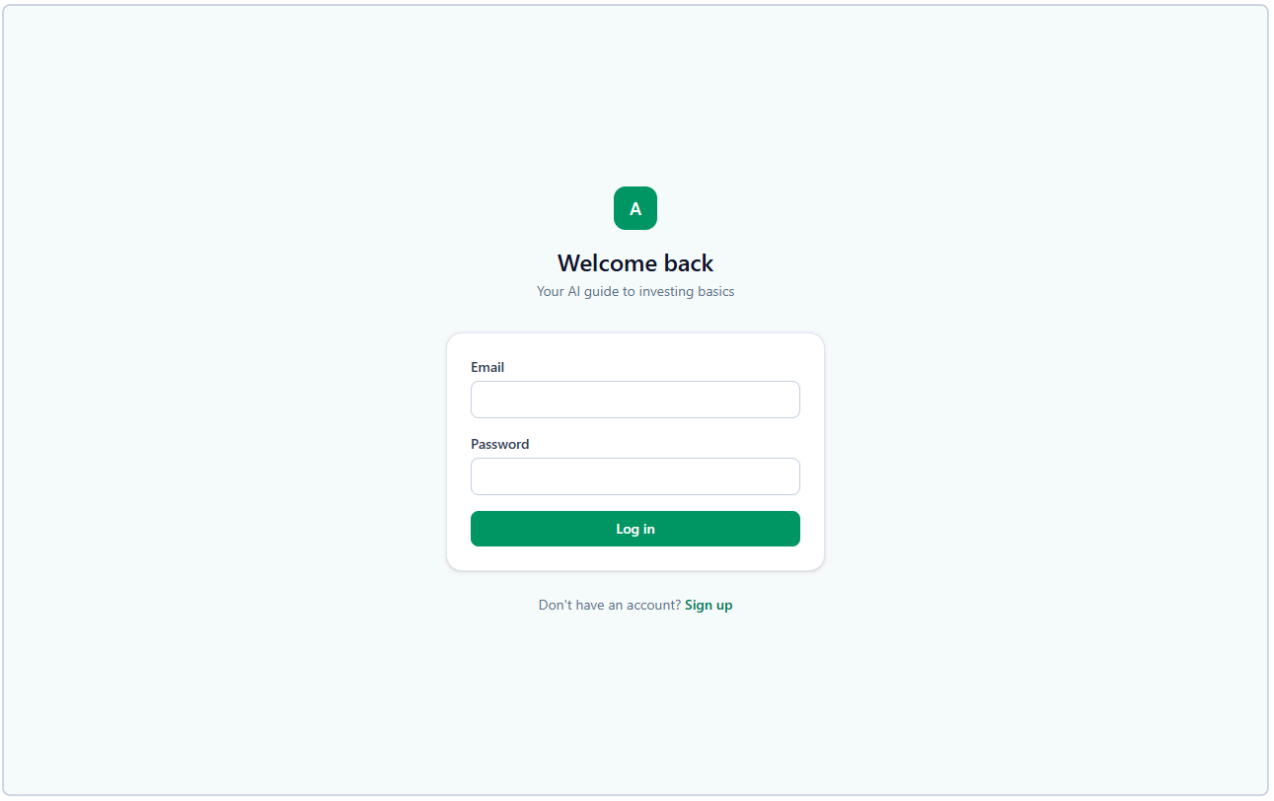
You don't need a frontend `.env` in development. `VITE_API_URL` only matters for production builds (see DEPLOYMENT.md).

Verify everything works

1. Health check:

   ```
   curl http://127.0.0.1:5000/api/health
   # {"status": "ok"}
   ```

2. Open http://localhost:5173. You should see the login page:



Login page

3. Click **Sign up**, create an account, and ask a question. A reply should stream in.
4. Optional end-to-end check from PowerShell, including a real Gemini call:

```
cd backend
.\scripts\smoke_test.ps1
```

5. Run the tests (see TESTING.md):

```
cd backend && pytest
cd frontend && npm test
```

Make yourself an admin

Register in the UI first, then from `backend/` with the venv active:

```
flask --app main make-admin you@example.com
# you@example.com is now an admin.
```

Log out and back in to see the **Admin** tab. Undo with `--revoke`.

Docker Compose

Runs nginx + the built frontend, gunicorn + Flask, Postgres 17 and Redis 7, like production.

```
cp backend/.env.example backend/.env
# set GEMINI_API_KEY, JWT_SECRET_KEY and SECRET_KEY in backend/.env
#   python -c "import secrets; print(secrets.token_hex(32))"
docker compose up --build
```

Open http://localhost:8080. Compose sets `FLASK_ENV=production`, so the backend refuses to start without `JWT_SECRET_KEY` (32+ characters) and `SECRET_KEY`. The database, Redis and proxy settings in `docker-compose.yml` override `.env`.

Make an admin inside the container:

```
docker compose exec backend flask --app main make-admin you@example.com
```

To test the Docker stack with the smoke test: `.\scripts\smoke_test.ps1 -BaseUrl http://localhost:8080/api`.

Common setup problems

Symptom	Fix
`venv\Scripts\Activate.ps1 cannot be loaded because running scripts is disabled`	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
Frontend shows "Cannot reach the server. Is the backend running?"	Start `python main.py`, or check `API_PROXY_TARGET`
Chat replies fail with "GEMINI_API_KEY is not configured"	Set it in `backend/.env` and restart the backend
`Address already in use` / port 5000 taken	See TROUBLESHOOTING.md

More in TROUBLESHOOTING.md.

See Also

- BACKEND.md and FRONTEND.md for the codebases
- DEPLOYMENT.md for going live

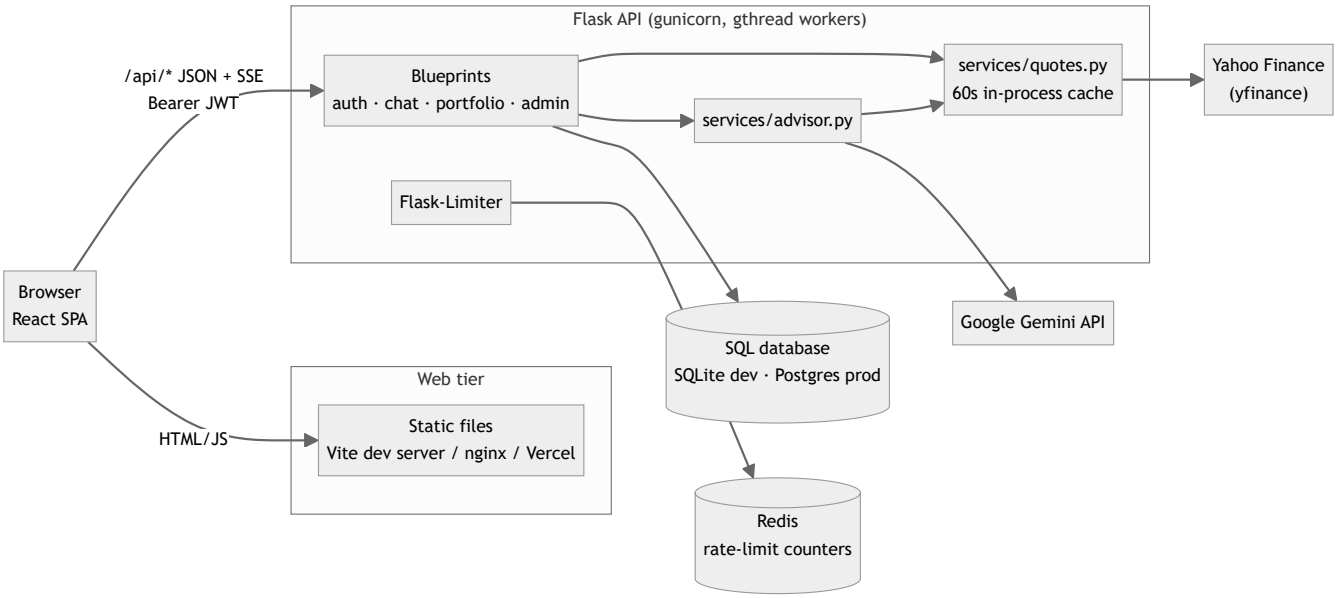
3 Architecture

Purpose: How the system fits together: components, data model, and what happens on each important request.

Overview

The app is a React single-page app talking to a Flask JSON API. The API stores users, chats and portfolios in a SQL database, calls Google Gemini for advisor replies, and fetches live prices from Yahoo Finance. Rate-limit counters live in Redis in production.

System diagram

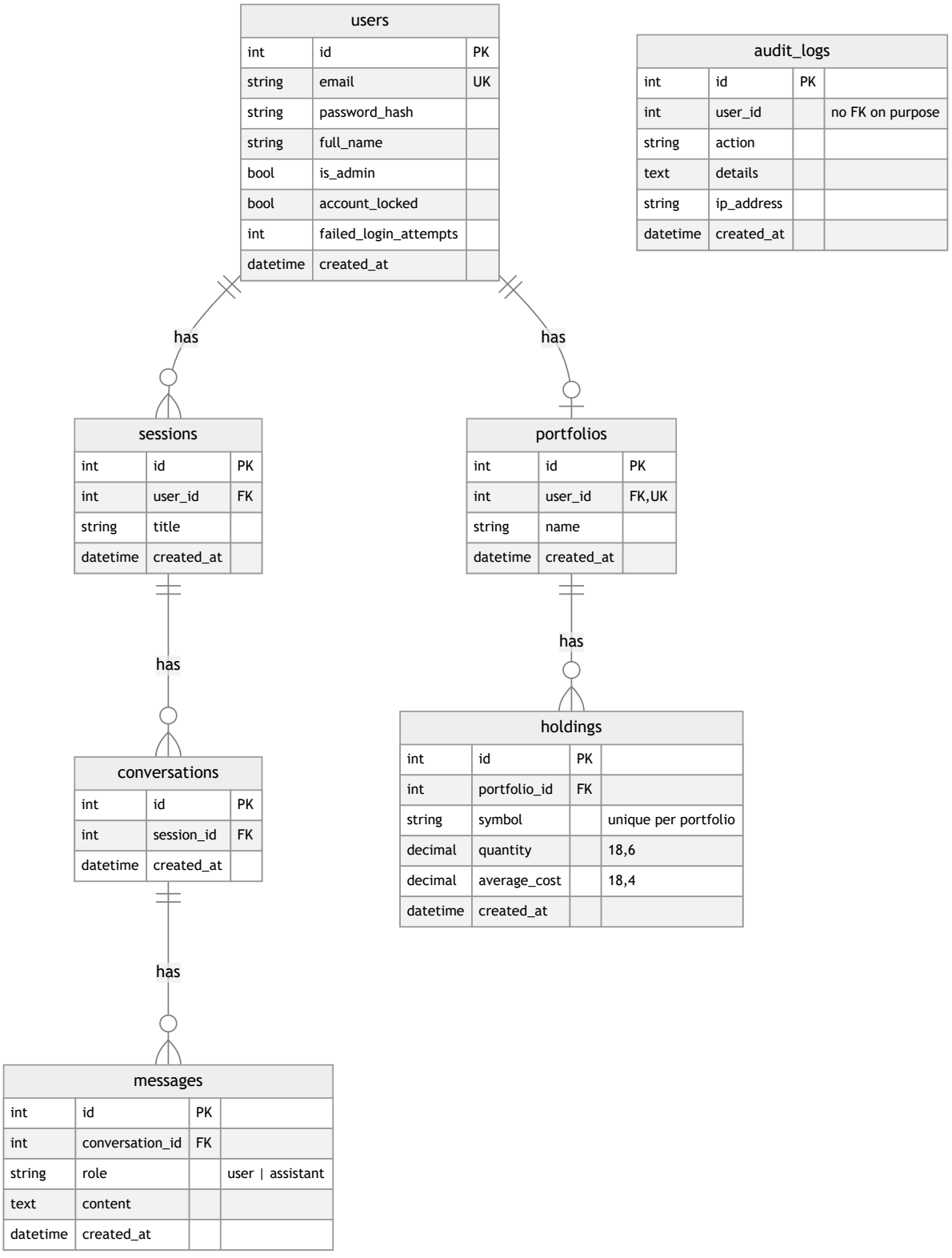


Environments

	Development	Docker Compose	Production (planned)
Frontend	Vite dev server, `:5173`	nginx container, `:8080`	Vercel
How `/api` is reached	Vite proxy → `127.0.0.1:5000`	nginx `proxy_pass` → `backend:5000`	`VITE_API_URL` → Render URL (cross-origin, CORS)
Backend	`python main.py` (Flask debug server)	gunicorn	gunicorn on Render
Database	SQLite `backend/instance/dev.db`	Postgres 17 container	Supabase Postgres
Rate-limit storage	`memory://`	Redis container	Render Key Value (Redis)
`TRUSTED_PROXY_COUNT`	0	1	1

There is no separate staging environment.

Database schema



Notes:

- Every parent → child relationship uses `cascade="all, delete-orphan"`, so deleting a chat session removes its conversations and messages.
- `audit_logs.user_id` is a plain integer with no foreign key, so audit history survives if a user is deleted.

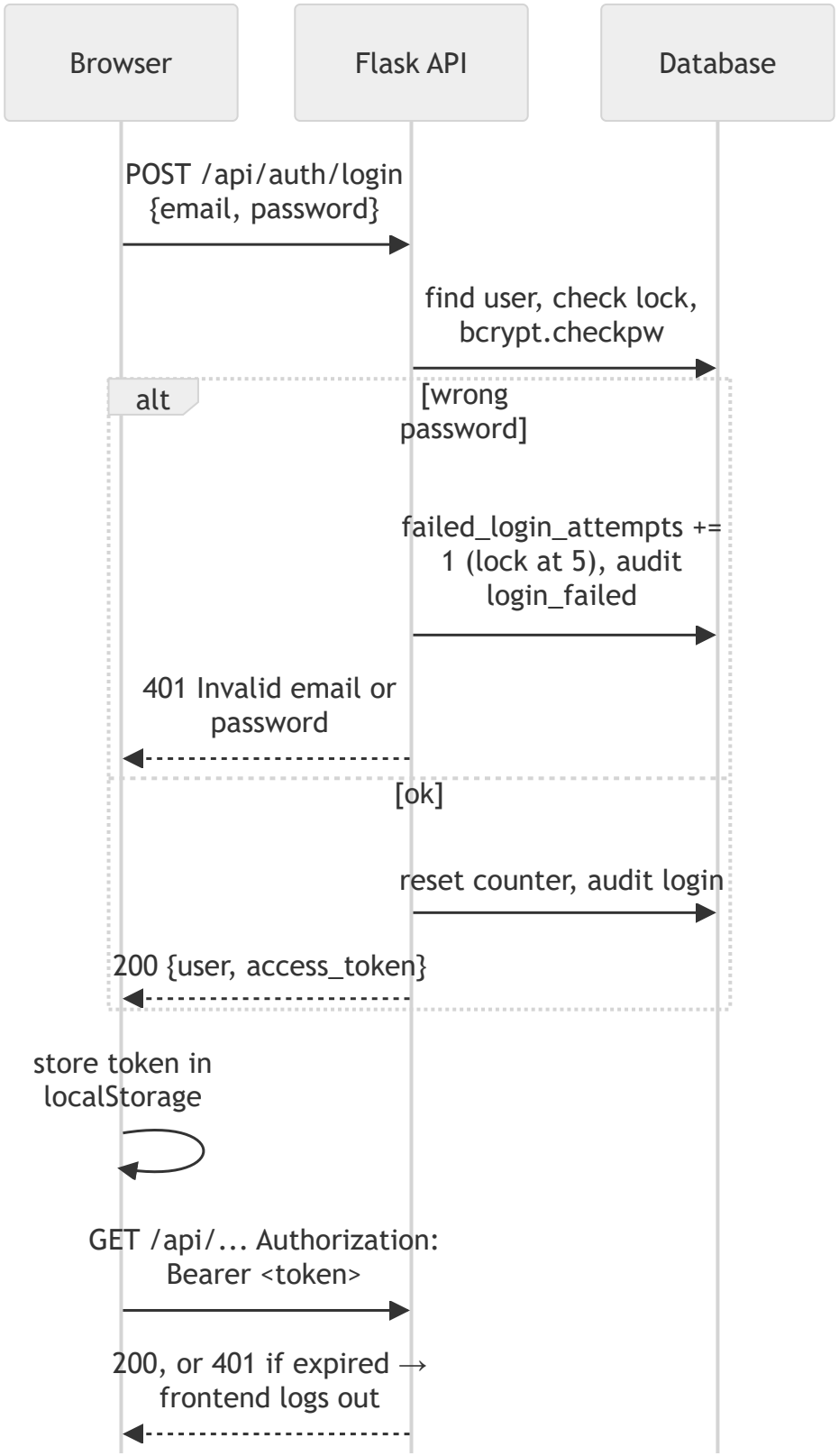
- The session model is called `ChatSession` in Python (table `sessions`) to avoid confusion with SQLAlchemy's `db.session`.
- A portfolio is created lazily the first time a user opens it or adds a holding. Each user has at most one.
- The UI currently uses one conversation per session. The schema allows several, which is why chat endpoints take a `conversation_id`.
- Schema changes go through Alembic migrations in `backend/migrations/`. `tests/test_migrations.py` fails if models and migrations drift apart.

API overview

All routes are under `/api`. Full details with examples are in API.md.

Prefix	Endpoints	Auth
`/api/health`	health check	none
`/api/auth`	register, login, me	none for register/login, JWT for `me`
`/api/chat`	create/list/delete sessions, send message (plain or streamed), history	JWT
`/api/portfolio`	get portfolio, live quotes, add/update/delete holding	JWT
`/api/admin`	list users, unlock user, audit logs	JWT + `is_admin`

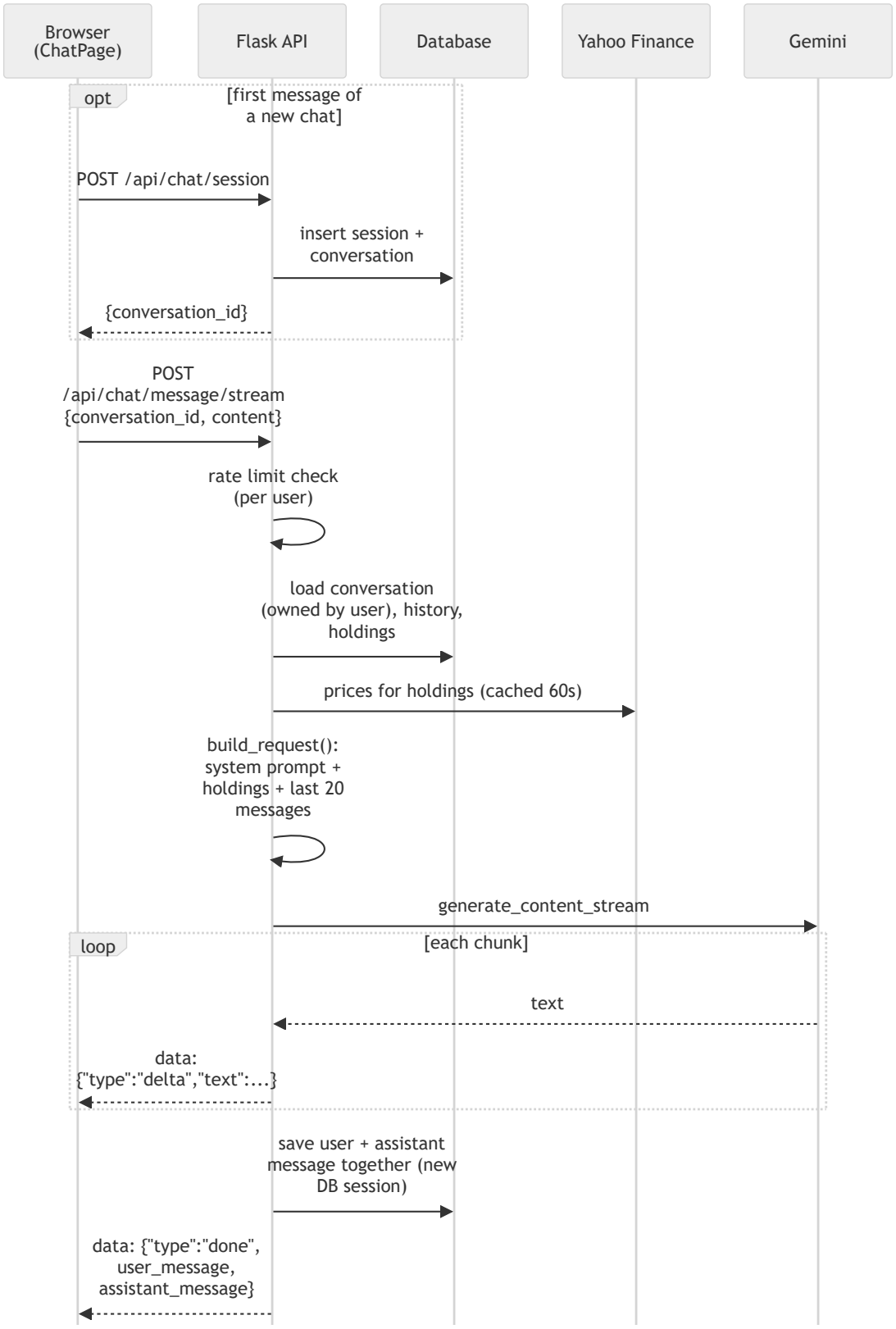
Authentication flow



Tokens are HS256 JWTs signed with `JWT_SECRET_KEY`, with the user ID (as a string) as the subject. They expire after 1 hour and there is no refresh token: the user logs in again. See SECURITY.md.

Chat data flow

The frontend uses the streaming endpoint. What happens when a user sends a message:

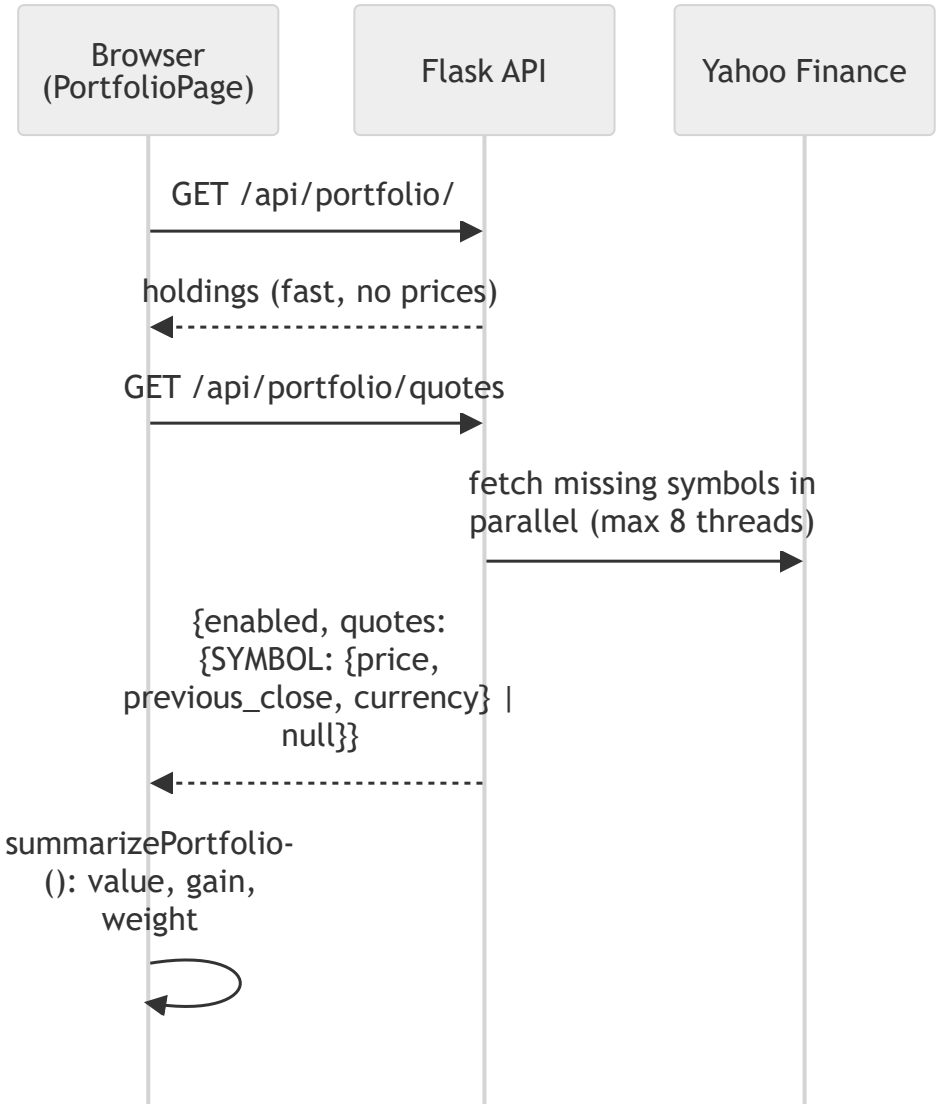


Key points:

- **All or nothing.** The user's message and the reply are committed in one transaction after the reply finishes. If Gemini fails, nothing is saved and the client receives an `error` event; the frontend puts the typed text back in the input box.

- **The database session closes early.** A real WSGI server closes the request's DB session before the streaming generator runs, so `stream_message` reads everything it needs (history, holdings, prices) up front through `advisor.build_request`, and saves in a fresh session at the end.
- **Retries.** Gemini 429 and 5xx errors are retried after 1s and 3s (`GEMINI_RETRY_DELAYS`). A stream is retried only if no text has reached the user yet.
- **History cap.** Only the last `CHAT_HISTORY_LIMIT` (20) messages are sent to Gemini.
- **Session titles.** An untitled session is named after the first 60 characters of its first message.

Portfolio and prices



Holdings and prices are separate calls so the table appears immediately while prices load. Unknown symbols or Yahoo failures return `null` for that symbol, and the UI values those holdings at cost. Portfolio math happens in the browser (`frontend/src/portfolio.ts`).

Caching and shared state

What	Where it lives	Notes
Stock prices	In-process dict, 60s TTL	Per gunicorn worker; not shared. Fine for this load.

What	Where it lives	Notes
Rate-limit counters	`RATELIMIT_STORAGE_URI`	`memory://` in dev. Must be Redis with several workers or instances, otherwise each worker has its own allowance.
Gemini client	Module-level singleton	Created on first use.
JWT	Browser `localStorage` (`advisor_token`)	Stateless on the server.

See Also

- BACKEND.md for the code behind these flows
- FRONTEND.md for the client side
- SECURITY.md for auth details

4 Backend

Purpose: Guide to the Flask API codebase in `backend/`: where things are, how they work, and how to do common tasks.

Overview

The backend is a Flask 3 app built with an app factory. It uses Flask-SQLAlchemy with Alembic migrations, Flask-JWT-Extended for auth, marshmallow for input validation, Flask-Limiter for rate limits, the `google-genai` SDK for Gemini, and `yfinance` for prices. In production it runs under gunicorn.

Project structure

```
backend/
├── main.py                 # Entry point: create_app(); `python main.py` migrates then runs the dev
server
├── config.py               # Development / Production / Testing config classes
├── requirements.txt
├── .env.example            # Copy to .env
├── Dockerfile              # python:3.13-slim, runs as non-root appuser
├── docker-entrypoint.sh    # db upgrade, then gunicorn (gthread)
├── app/
│   ├── __init__.py         # create_app(), db, jwt, limiter, migrate, error handlers, make-admin CLI
│   ├── models/__init__.py  # All SQLAlchemy models
│   ├── schemas/__init__.py # marshmallow request schemas
│   ├── middleware/__init__.py  # current_user_id, admin_required, rate-limit helpers
│   ├── routes/
│   │   ├── auth.py         # /api/auth
│   │   ├── chat.py         # /api/chat
│   │   ├── portfolio.py    # /api/portfolio
│   │   └── admin.py        # /api/admin
│   └── services/
│       ├── advisor.py      # Gemini prompt building, calls, retries, streaming
│       ├── quotes.py       # Yahoo Finance prices with a 60s cache
│       └── audit.py        # record_audit()
├── migrations/             # Alembic (Flask-Migrate)
├── scripts/
│   ├── chat.ps1            # Interactive terminal chat client
│   └── smoke_test.ps1      # End-to-end check against a running server
└── tests/                  # pytest
```

App factory

`app/__init__.py` `create_app(config_name=None)`:

1. Picks the config class from `FLASK_ENV` (default `development`) and runs its `validate()` if it has one.

2. Wraps the app in `ProxyFix` when `TRUSTED_PROXY_COUNT > 0`, so `request.remote_addr` is the real client IP behind nginx or Render.
3. Initialises `db`, `migrate` (with `render_as_batch=True` so migrations can alter columns on SQLite), `jwt`, `limiter` and CORS.
4. Registers the four blueprints and `GET /api/health`.
5. Registers JSON error handlers and the `make-admin` CLI command.

Configuration

All settings are in `config.py` and read from environment variables (loaded from `backend/.env` by python-dotenv). A blank value in `.env` counts as unset.

Variable	Default	Meaning
`FLASK_ENV`	`development`	`development`, `production` or `testing`
`DATABASE_URL`	`sqlite:///dev.db`	`postgres://` and `postgresql://` are rewritten to `postgresql+psycopg://`
`GEMINI_API_KEY`	none	Required for chat
`GEMINI_MODEL`	`gemini-2.5-flash`	
`GEMINI_MAX_OUTPUT_TOKENS`	`8192`	Includes 2.5 models' thinking tokens, so keep it generous
`JWT_SECRET_KEY`	dev value	Required, 32+ chars, in production
`SECRET_KEY`	dev value	Required in production
`CORS_ORIGINS`	`http://localhost:5173`	Comma-separated
`CHAT_RATE_LIMIT`	`10 per minute;200 per day`	Per user, shared by both message endpoints
`AUTH_RATE_LIMIT`	`10 per minute`	Per IP, login and register
`RATELIMIT_STORAGE_URI`	`memory://`	Use `redis://...` with several workers
`TRUSTED_PROXY_COUNT`	`0`	Number of reverse proxies in front
`QUOTES_ENABLED`	`true`	Turn live prices off

Fixed in code: token lifetime 1 hour, `BCRYPT_ROUNDS = 12`, `MAX_FAILED_LOGINS = 5`, `CHAT_HISTORY_LIMIT = 20`, `GEMINI_RETRY_DELAYS = (1, 3)`, `QUOTES_CACHE_SECONDS = 60`.

`ProductionConfig.validate()` refuses to start without `JWT_SECRET_KEY`, `SECRET_KEY` and `DATABASE_URL`. `TestingConfig` uses in-memory SQLite, 4 bcrypt rounds, no quotes and no retry delays.

Models

All in `app/models/__init__.py`. See the diagram in ARCHITECTURE.md.

Model	Table	Notes
`User`	`users`	`to_dict()` never includes `password_hash`
`ChatSession`	`sessions`	`to_dict()` includes `conversation_ids`
`Conversation`	`conversations`	`messages` ordered by `Message.id`
`Message`	`messages`	`role` is `"user"` or `"assistant"`

Model	Table	Notes
`Portfolio`	`portfolios`	One per user (`user_id` unique)
`Holding`	`holdings`	Unique `(portfolio_id, symbol)` ; quantity and cost are `Numeric` , serialised as strings
`AuditLog`	`audit_logs`	No relationship to `User` on purpose

Timestamps use the `utcnow()` helper (timezone-aware UTC).

Routes

File	Endpoints
`routes/auth.py`	`POST /register` , `POST /login` , `GET /me`
`routes/chat.py`	`POST /session` , `GET /sessions` , `POST /message` , `POST /message/stream` , `GET /history/<id>` , `DELETE /session/<id>`
`routes/portfolio.py`	`GET /` , `GET /quotes` , `POST /holding` , `PUT /holding/<id>` , `DELETE /holding/<id>`
`routes/admin.py`	`GET /users` , `POST /users/<id>/unlock` , `GET /audit-logs`

Request and response details: API.md.

The chat message endpoints share `_start_exchange` / `_finish_exchange` , which stage the user's message and commit it together with the reply. `stream_message` can't use the request's DB session inside its generator (see ARCHITECTURE.md), so it builds the Gemini request first and re-loads the conversation by ID when saving.

Services

`services/advisor.py`

- `SYSTEM_PROMPT` : frames the model as a financial *education* assistant that is not a licensed advisor.
- `build_request(messages, portfolio)` : turns history (last 20 messages) and holdings with live prices into `generate_content` arguments. Stored role `assistant` becomes Gemini's `model` .
- `generate_reply(messages, portfolio)` : one-shot reply, used by `POST /message` .
- `stream_reply(request)` : yields text chunks, used by `POST /message/stream` .
- Retries 429/500/502/503/504 with `GEMINI_RETRY_DELAYS` . Streams retry only before any text was sent.
- If Gemini stops at the token limit, appends "*(This answer was cut off because it hit the length limit.)*".
- All failures raise `AdvisorUnavailableError` with a message safe to show users. Routes turn it into a 503 (plain) or an `error` event (stream).

`services/quotes.py`

- `get_quotes(symbols)` returns `{symbol: {"price", "previous_close", "currency"} | None}` and never raises.
- Missing symbols are fetched in parallel (up to 8 threads) and cached for 60 seconds per process.
- Returns `{}` when `QUOTES_ENABLED=false` .
- To switch to a licensed data provider, replace `_fetch_quote` .

`services/audit.py`

`record_audit(action, user_id=None, details=None)` adds an `AuditLog` row with the client IP to the current transaction. The caller commits.

Middleware

`app/middleware/__init__.py`:

- `current_user_id()`: the JWT subject as an `int`. Use this, not `get_jwt_identity()` directly.
- `@admin_required`: verifies the JWT and that the user has `is_admin`, else 403.
- `user_rate_limit_key()`: `user:<id>` when logged in, else client IP.
- `chat_rate_limit()` / `auth_rate_limit()`: read the limit strings from config at request time.

Error handling

Every error response is JSON `{"error": "..."}`:

Source	Status	Body
marshmallow `ValidationError`	400	`{"error": "Invalid input", "details": {field: [messages]}}`
`abort(code, description=...)`	code	`{"error": description}`
Rate limit	429	`{"error": "Too many requests (10 per 1 minute). Please wait a moment and try again."}`
JWT missing / invalid / expired	401	`{"error": "..."}`
Unhandled exception	500	`{"error": "Internal server error"}`, logged with traceback, session rolled back

Conventions

- **Ownership:** every query for a user-owned resource filters by the JWT user, joining through parents (`_get_owned_conversation`, `_get_owned_holding`). Another user's resource returns **404, not 403**, so IDs can't be probed.
- **Validation:** `XSchema().load(request.get_json(silent=True) or {})`.
- **Audit:** call `record_audit(...)` before `db.session.commit()` so the log and the change are saved together.
- **Tests never call Gemini or Yahoo** (see TESTING.md).

Common tasks

Add a model field

```
# 1. edit app/models/__init__.py
flask --app main db migrate -m "add risk_profile to users"
# 2. review the file in migrations/versions/, then
flask --app main db upgrade
pytest tests/test_migrations.py
```

Commit the migration file. Deploys apply it automatically (`docker-entrypoint.sh` runs `db upgrade`).

Add an endpoint

```
# app/routes/portfolio.py
@portfolio_bp.get("/summary")
@jwt_required()
def summary():
    portfolio = _get_or_create_portfolio(current_user_id())
    db.session.commit()
    return jsonify({"holdings": len(portfolio.holdings)})
```

Add a marshmallow schema in `app/schemas/` for any request body, a test in `tests/`, and an entry in API.md.

Chat from the terminal

```
.\scripts\chat.ps1
```

Grant or revoke admin

```
flask --app main make-admin you@example.com
flask --app main make-admin you@example.com --revoke
```

See Also

- ARCHITECTURE.md for the request flows
- API.md for endpoint details
- SECURITY.md for auth and rate limiting

5 Frontend

Purpose: Guide to the React app in `frontend/`: structure, routing, state, streaming and styling.

Overview

A React 19 + TypeScript single-page app built with Vite 8, routed with React Router 8 and styled with Tailwind CSS v4. It has four screens (auth, chat, portfolio, admin) and no global state library: auth state lives in a React context and each page keeps its own state.

File structure

```
frontend/
├── index.html
├── vite.config.ts          # React + Tailwind plugins, /api dev proxy
├── vitest.config.ts        # jsdom test environment
├── nginx.conf              # Used by the Docker image: static files + /api proxy
├── vercel.json             # SPA rewrite for Vercel
├── Dockerfile              # node:24 build → nginx:1.29
└── src/
    ├── main.tsx            # Mounts <App />
    ├── App.tsx             # Routes per role
    ├── api.ts              # Typed API client, ApiError, SSE parsing, streamMessage
    ├── auth.tsx            # AuthProvider / useAuth
    ├── portfolio.ts        # summarizePortfolio(): value, gain, weight math
    ├── index.css           # Tailwind v4 config (@import "tailwindcss", typography plugin)
    ├── components/
    │   └── Layout.tsx      # Top bar with nav and log out
    ├── pages/
    │   ├── AuthPage.tsx    # Login and register (mode prop)
    │   ├── ChatPage.tsx    # Sidebar of chats, messages, streaming input
    │   ├── PortfolioPage.tsx
    │   └── AdminPage.tsx   # Users tab and audit log tab
    └── test/
        ├── setup.ts
        └── fakeServer.ts   # fetch stub for tests
```

There are no separate `hooks/` or `utils/` folders; small helpers live next to the component that uses them.

Routing

`App.tsx` renders a different route set depending on who is logged in, instead of guarding each route:

State	Routes	Anything else
Checking token	"Loading…" screen	
Logged out	`/login`, `/register`	→ `/login`

State	Routes	Anything else
User	`/chat/:conversationId?`, `/portfolio`	→ `/chat`
Admin	the above + `/admin`	→ `/chat`

Chat uses **one route with an optional parameter** so the page doesn't remount when a new chat receives its ID and the URL changes from `/chat` to `/chat/12`.

API client

All HTTP goes through `src/api.ts`:

- `api` object with one typed method per endpoint (`api.login`, `api.portfolio`, `api.streamMessage`, …).
- Base URL is `import.meta.env.VITE_API_URL ?? '/api'`.
- Adds `Authorization: Bearer <token>` when a token is stored.
- Throws `ApiError(status, message, details)` for non-2xx responses and network failures. `err.friendlyMessage` gives the first field validation message, e.g. `"password: Length must be between 8 and 128."`.
- A 401 while logged in calls the handler registered by `AuthProvider`, which logs the user out and shows "Your session expired".

```
try {
  const { holding } = await api.addHolding('VTI', '10', '215.50')
} catch (err) {
  setError(err instanceof ApiError ? err.friendlyMessage : 'Could not add holding')
}
```

Authentication

`src/auth.tsx` provides `useAuth()` → `{ user, loading, sessionExpired, login, register, logout }`.

- The JWT is stored in `localStorage` under `advisor_token`.
- On page load, if a token exists, `GET /api/auth/me` validates it. If that fails, the token is cleared.
- `logout()` only clears local state; tokens are stateless and expire after 1 hour.

Chat page

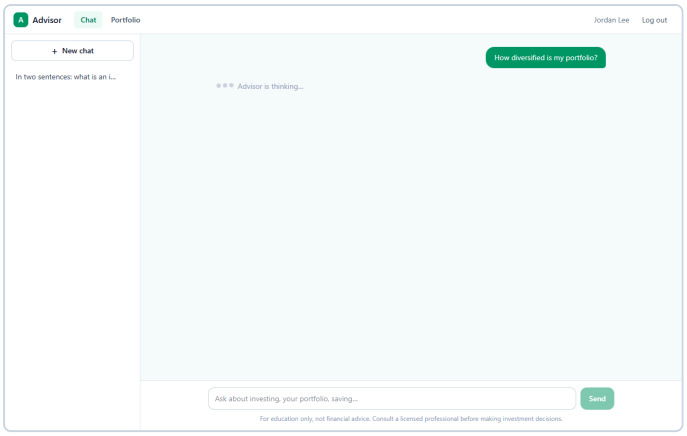
`src/pages/ChatPage.tsx`:

- **Sidebar** lists sessions (`GET /chat/sessions`) with a delete button each, and **New chat**. On mobile it becomes a drawer opened with ☰ **Conversations**.
- **Lazy session creation:** a session is created only when the first message is sent, so empty chats don't pile up.
- **Streaming:** `api.streamMessage(id, content, onDelta)` reads the SSE body with a `TextDecoderStream`, splits events with `parseSSE`, calls `onDelta` per chunk, and resolves on `done` or throws on `error`.
- **Optimistic message:** the user's bubble appears immediately with a temporary negative ID and is replaced by the saved messages on `done`. On error it is removed and the text goes back into the input.
- **Race protection:** `activeIdRef` holds the conversation currently on screen. It is set *before* `navigate()` because router navigation is a transition, and a fast reply could otherwise be treated as belonging to a

different chat and dropped.

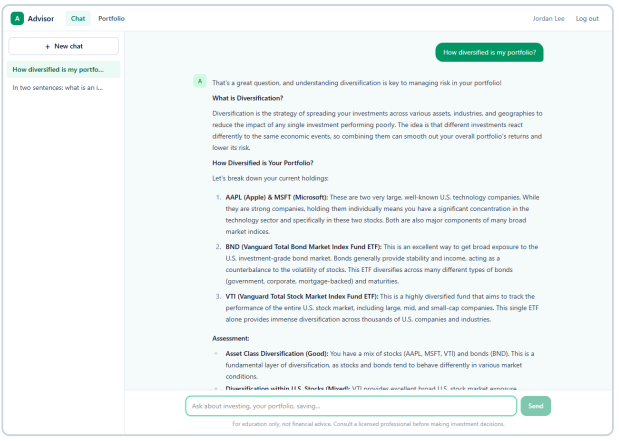
- Replies are rendered as Markdown with `react-markdown` and `prose` classes.
- **Enter** sends; **Shift+Enter** inserts a newline. Max 4000 characters.

Waiting



Thinking indicator

Answer

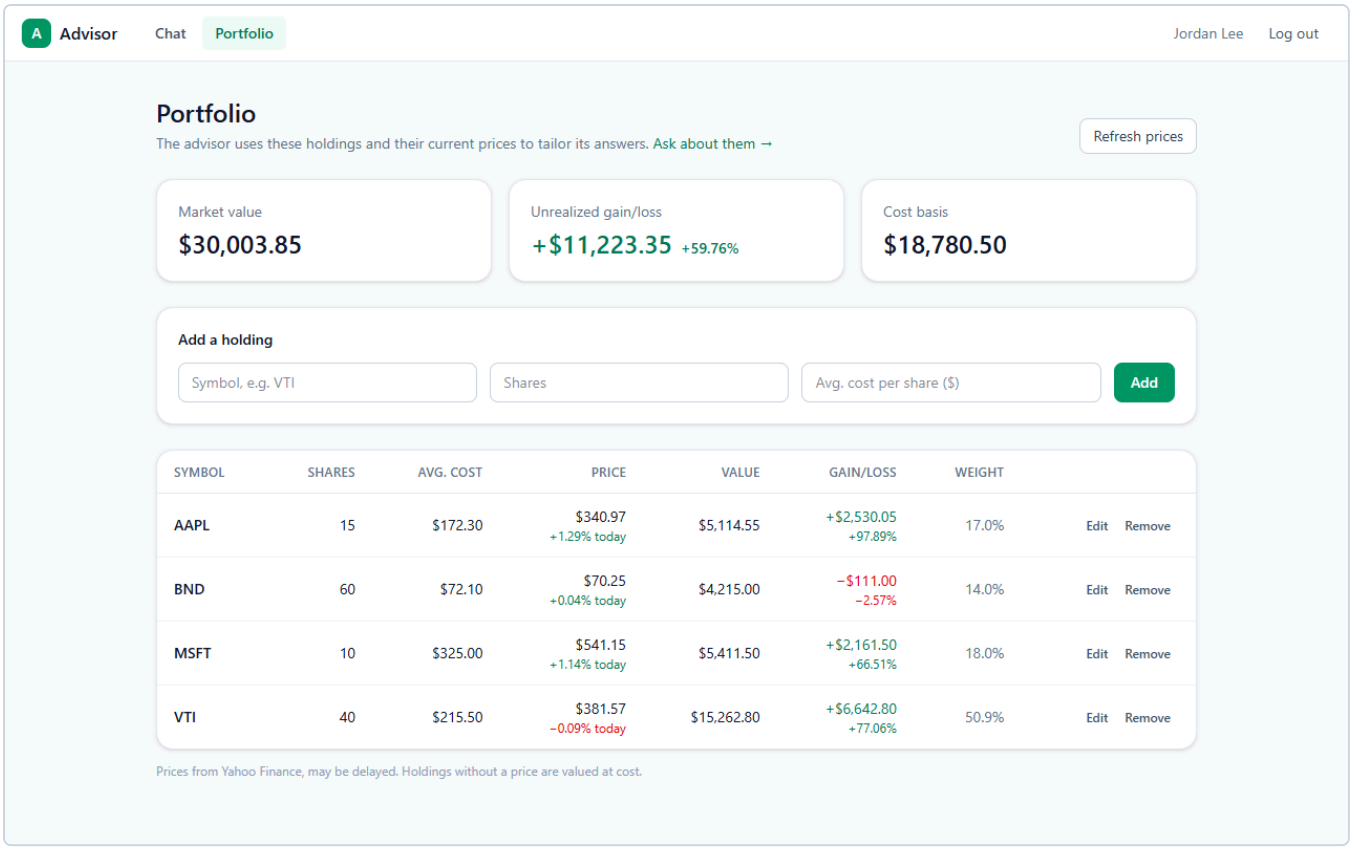


Streamed answer

Portfolio page

`src/pages/PortfolioPage.tsx`:

- Loads holdings (`GET /portfolio/`) and prices (`GET /portfolio/quotes`) separately.
- Re-fetches prices only when the *set of symbols* changes, not on quantity edits.
- `latestQuoteRequest` ref ignores out-of-order quote responses, so a slow old response can't overwrite a newer one.
- `summarizePortfolio()` in `src/portfolio.ts` computes per-row value, gain, gain %, day change and weight, and totals. Holdings without a price are valued at cost and excluded from gain.
- Rows support inline edit and two-step delete (Remove → Delete / Keep).



Portfolio page

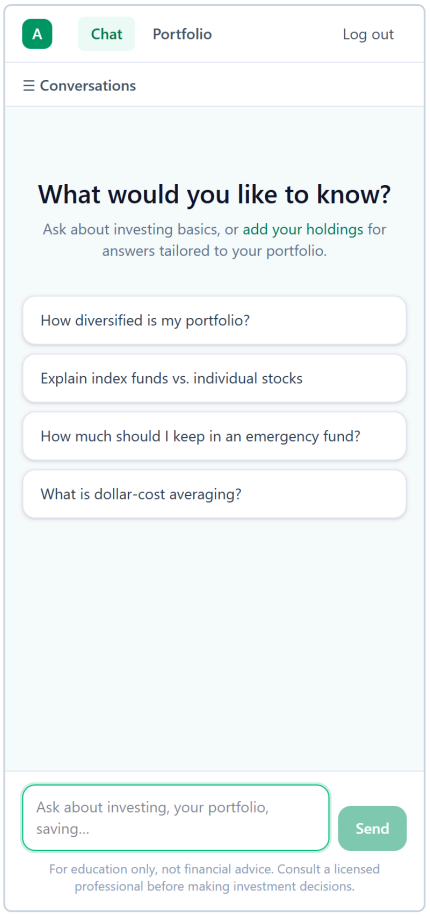
Admin page

`src/pages/AdminPage.tsx` has two tabs, each paged 50 per page by the shared `usePaged` hook:

- **Users:** email, name, join date, status, **Unlock** for locked accounts.
- **Audit log:** filter by action; failure and lock events are highlighted.

Styling

- Tailwind CSS v4 via `@tailwindcss/vite`, configured in `src/index.css` (no `tailwind.config.js`).
- `@tailwindcss/typography` (`prose` classes) for Gemini's Markdown.
- Palette: slate neutrals with emerald as the accent color.
- Responsive: the chat sidebar collapses below the `md` breakpoint and the top bar hides the name below `md`.



Mobile chat

Build and environment

```
npm run dev      # dev server on :5173 with /api proxy
npm run build    # tsc -b (includes tests) then vite build → dist/
npm run lint     # oxlint
npm test         # vitest
npm run preview  # serve dist/ locally
```

Variable	When	Meaning
`VITE_API_URL`	build time	API base URL, e.g. `https://advisor-api-xxxx.onrender.com/api`. Default `/api`.
`API_PROXY_TARGET`	dev server	Where `/api` is proxied. Default `http://127.0.0.1:5000`.

`VITE_API_URL` is baked into the bundle at build time, so changing it requires a rebuild.

See Also

- API.md for the endpoints the client calls
- TESTING.md for the frontend tests
- USER_GUIDE.md for the screens from a user's point of view

6 API Reference

Purpose: Every endpoint of the Flask API with request bodies, real response examples, status codes and curl commands.

Overview

The API is JSON over HTTP under `/api`. Authenticated endpoints need `Authorization: Bearer <access_token>` from login or register. Every error is `{"error": "..."}`, with a `details` object added for validation errors.

The response examples below are real responses captured from the running app on 2026-10-13 (tokens shortened). Timestamps from the development SQLite database have no UTC offset; on Postgres they end in `+00:00`.

Conventions

```
BASE=http://127.0.0.1:5000/api      # dev; Docker: http://localhost:8080/api
TOKEN=<access_token from login>
```

Status	Meaning
200 / 201 / 204	OK / created / deleted (empty body)
400	Validation failed: `{"error": "Invalid input", "details": {...}}`
401	Missing, invalid or expired token, or wrong credentials
403	Account locked, or admin required
404	Not found, **or owned by another user**
409	Duplicate (email or symbol)
429	Rate limited. Includes `Retry-After` and `X-RateLimit-*` headers
503	Gemini unavailable (plain message endpoint only)

Rate-limited response (captured after 11 rapid logins):

```
HTTP/1.1 429 TOO MANY REQUESTS
X-RateLimit-Limit: 10
X-RateLimit-Remaining: 0
Retry-After: 59

{"error": "Too many requests (10 per 1 minute). Please wait a moment and try again."}
```

Health

`GET /api/health`

No auth, not rate limited. Used by Docker and Render health checks.

```
curl $BASE/health
```

```
{"status": "ok"}
```

Auth

`POST /api/auth/register`

Rate limited per IP (`AUTH_RATE_LIMIT`). Returns a token, so the user is logged in immediately.

Field	Type	Rules
`email`	string	required, valid email, stored lowercase
`password`	string	required, 8–128 characters
`full_name`	string	optional, max 120

```
curl -X POST $BASE/auth/register -H "Content-Type: application/json" \
  -d '{"email":"jordan@example.com","password":"demo-pass-123","full_name":"Jordan Lee"}'
```

`201 Created`

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.…",
  "user": {
    "account_locked": false,
    "created_at": "2026-10-13T04:50:59.155017",
    "email": "jordan@example.com",
    "full_name": "Jordan Lee",
    "id": 1,
    "is_admin": false
  }
}
```

`400` for invalid input:

```
{
  "details": {
    "email": ["Not a valid email address."],
    "password": ["Length must be between 8 and 128."]
  },
  "error": "Invalid input"
}
```

`409` `{"error": "Email is already registered"}`

`POST /api/auth/login`

Rate limited per IP. After 5 wrong passwords in a row the account is locked until an admin unlocks it.

```
curl -X POST $BASE/auth/login -H "Content-Type: application/json" \
  -d '{"email":"jordan@example.com","password":"demo-pass-123"}'
```

`200 OK` : same shape as register.

Status	Body
401	`{"error": "Invalid email or password"}` (same for unknown email and wrong password)
403	`{"error": "Account is locked"}`

`GET /api/auth/me`

```
curl $BASE/auth/me -H "Authorization: Bearer $TOKEN"
```

`200 OK`

```
{
  "user": {
    "account_locked": false,
    "created_at": "2026-10-13T04:50:59.155017",
    "email": "jordan@example.com",
    "full_name": "Jordan Lee",
    "id": 1,
    "is_admin": false
  }
}
```

Without a token: `401` `{"error": "Missing Authorization Header"}`. Expired: `401` `{"error": "Token has expired"}`.

Chat

All chat endpoints require a token. Users can only see their own sessions and conversations; anything else returns 404.

`POST /api/chat/session`

Creates a session and its first conversation. Body is optional: `{"title": "..."}` (max 200). Untitled sessions are named after their first message.

```
curl -X POST $BASE/chat/session -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{}'
```

`201 Created`

```
{
  "conversation_id": 1,
  "session": {
    "conversation_ids": [1],
    "created_at": "2026-10-13T04:51:08.698829",
    "id": 1,
    "title": null
  }
}
```


`GET /api/chat/sessions`

Newest first.


```
{
  "sessions": [
    {
      "conversation_ids": [2],
      "created_at": "2026-10-13T04:52:07.948115",
      "id": 2,
      "title": "How diversified is my portfolio?"
    },
    {
      "conversation_ids": [1],
      "created_at": "2026-10-13T04:51:08.698829",
      "id": 1,
      "title": "In two sentences: what is an index fund?"
    }
  ]
}
```


`POST /api/chat/message`

Sends a message and waits for the full reply. Rate limited per user (`CHAT_RATE_LIMIT`, shared with the streaming endpoint). Both messages are saved together; if Gemini fails nothing is saved.

Field	Type	Rules
`conversation_id`	int	required, must belong to you
`content`	string	required, 1–4000 characters

```
curl -X POST $BASE/chat/message -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"conversation_id":1,"content":"In two sentences: what is an index fund?"}'
```

`201 Created` (real Gemini reply)

```
{
  "assistant_message": {
    "content": "An index fund is a type of mutual fund or Exchange Traded Fund (ETF) designed to track the performance of a specific market index, like the S&P 500 or a total stock market index. By holding a diversified collection of investments that mirror the chosen index, it offers broad market exposure and generally comes with lower fees compared to actively managed funds.",
    "created_at": "2026-10-13T04:51:27.606159",
    "id": 2,
    "role": "assistant"
  },
  "user_message": {
    "content": "In two sentences: what is an index fund?",
    "created_at": "2026-10-13T04:51:21.626647",
    "id": 1,
    "role": "user"
  }
}
```


Status	Body
404	`{"error": "Conversation not found"}`
503	e.g. `{"error": "The advisor is getting too many requests right now. Please wait a minute and try again."}`

`POST /api/chat/message/stream`

Same body and rate limit as `/message`, but the response is `text/event-stream`. Each event is one `data: <json>` line followed by a blank line:

Event	Fields	When
`delta`	`text`	Each chunk of the reply
`done`	`user_message`, `assistant_message`	Both messages saved; stream ends
`error`	`error`	Gemini failed; nothing saved; stream ends

Errors before the stream starts (validation, 404, 429) come back as normal JSON responses.

```
curl -N -X POST $BASE/chat/message/stream -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"conversation_id":1,"content":"In one sentence: what is a bond?"}'
```

Real output:

```
data: {"type": "delta", "text": "A bond is essentially a loan made by an investor to a borrower—typically a corporation or government—in"}

data: {"type": "delta", "text": " exchange for periodic interest payments and the return of the principal amount on a specified maturity date."}

data: {"type": "done", "user_message": {"id": 3, "role": "user", "content": "In one sentence: what is a bond?", "created_at": "2026-10-13T04:51:27.729644"}, "assistant_message": {"id": 4, "role": "assistant", "content": "A bond is essentially a loan made by an investor to a borrower—typically a corporation or government—in exchange for periodic interest payments and the return of the principal amount on a specified maturity date.", "created_at": "2026-10-13T04:51:29.318128"}}
```

`GET /api/chat/history/<conversation_id>`

```
{
  "conversation_id": 1,
  "messages": [
    {"id": 1, "role": "user", "content": "In two sentences: what is an index fund?", "created_at": "..."},
    {"id": 2, "role": "assistant", "content": "An index fund is ...", "created_at": "..."}
  ]
}
```

`404` `{"error": "Conversation not found"}` for unknown IDs or another user's conversation.

`DELETE /api/chat/session/<session_id>`

Deletes the session with its conversations and messages. `204 No Content`, or `404` `{"error": "Session not found"}`.

Portfolio

All portfolio endpoints require a token. The portfolio is created on first access.

`GET /api/portfolio/`

Note the trailing slash. Quantities and costs are decimal strings.

```
curl $BASE/portfolio/ -H "Authorization: Bearer $TOKEN"
```

```
{
  "portfolio": {
    "holdings": [
      {"average_cost": "172.3000", "id": 2, "quantity": "15.000000", "symbol": "AAPL"},
      {"average_cost": "72.1000",  "id": 3, "quantity": "60.000000", "symbol": "BND"},
      {"average_cost": "325.0000", "id": 4, "quantity": "10.000000", "symbol": "MSFT"},
      {"average_cost": "215.5000", "id": 1, "quantity": "40.000000", "symbol": "VTI"}
    ],
    "id": 1,
    "name": "My Portfolio"
  }
}
```


`GET /api/portfolio/quotes`

Live prices for your holdings from Yahoo Finance, cached 60 seconds. A symbol Yahoo doesn't know maps to `null`. `enabled` is `false` when `QUOTES_ENABLED=false` (then `quotes` is `{}`).


```
{
  "enabled": true,
  "quotes": {
    "AAPL": {"currency": "USD", "previous_close": 336.64, "price": 340.97},
    "BND":  {"currency": "USD", "previous_close": 70.22,  "price": 70.25},
    "MSFT": {"currency": "USD", "previous_close": 535.07, "price": 541.15},
    "VTI":  {"currency": "USD", "previous_close": 381.9,  "price": 381.57}
  }
}
```


`POST /api/portfolio/holding`

Field	Type	Rules
`symbol`	string	required, 1–16 of `A-Z a-z . -`, stored uppercase
`quantity`	decimal	required, > 0
`average_cost`	decimal	required, ≥ 0

```
curl -X POST $BASE/portfolio/holding -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"symbol":"vti","quantity":"40","average_cost":"215.50"}'
```

`201 Created`


```
{"holding": {"average_cost": "215.5000", "id": 1, "quantity": "40.000000", "symbol": "VTI"}}
```


`409` `{"error": "VTI is already in the portfolio; update it instead"}`

`PUT /api/portfolio/holding/<holding_id>`

Body: `quantity` and `average_cost` (same rules). The symbol can't be changed.

```
curl -X PUT $BASE/portfolio/holding/4 -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"quantity":"10","average_cost":"325.00"}'
```

```
{"holding": {"average_cost": "325.0000", "id": 4, "quantity": "10.000000", "symbol": "MSFT"}}
```

`DELETE /api/portfolio/holding/<holding_id>`

`204 No Content`, or `404` `{"error": "Holding not found"}`.

Admin

Requires a token for a user with `is_admin`. Others get `403` `{"error": "Admin access required"}`. List endpoints take `page` (default 1) and `per_page` (default 50, max 200).

`GET /api/admin/users`

```
curl "$BASE/admin/users?page=1" -H "Authorization: Bearer $ADMIN_TOKEN"
```

`?per_page=2` (real output):

```
{
  "page": 1,
  "total": 3,
  "users": [
    {
      "account_locked": false,
      "created_at": "2026-10-13T04:50:59.155017",
      "email": "jordan@example.com",
      "full_name": "Jordan Lee",
      "id": 1,
      "is_admin": false
    },
    {
      "account_locked": false,
      "created_at": "2026-10-13T04:50:59.444654",
      "email": "admin@example.com",
      "full_name": "Site Admin",
      "id": 2,
      "is_admin": true
    }
  ]
}
```

`POST /api/admin/users/<user_id>/unlock`

Clears the lock and the failed-login counter, and writes an `account_unlocked` audit entry. Returns `{"user": {...}}`.

`GET /api/admin/audit-logs`

Newest first. Optional filters: `user_id`, `action`.

Actions: `register`, `login`, `login_failed`, `login_blocked_locked`, `account_unlocked`, `holding_added`, `holding_updated`, `holding_deleted`.

```
curl "$BASE/admin/audit-logs?action=login_failed&per_page=2" -H "Authorization: Bearer $ADMIN_TOKEN"
```


```
{
  "audit_logs": [
    {
      "action": "login_failed",
      "created_at": "2026-10-13T04:53:56.013372",
      "details": null,
      "id": 29,
      "ip_address": "127.0.0.1",
      "user_id": 1
    },
    {
      "action": "login_failed",
      "created_at": "2026-10-13T04:53:11.939765",
      "details": null,
      "id": 24,
      "ip_address": "127.0.0.1",
      "user_id": 1
    }
  ],
  "page": 1,
  "total": 9
}
```


Endpoint summary

Method	Path	Auth	Rate limit
GET	`/api/health`	none	exempt
POST	`/api/auth/register`	none	per IP
POST	`/api/auth/login`	none	per IP
GET	`/api/auth/me`	JWT	
POST	`/api/chat/session`	JWT	
GET	`/api/chat/sessions`	JWT	
POST	`/api/chat/message`	JWT	per user (shared)
POST	`/api/chat/message/stream`	JWT	per user (shared)
GET	`/api/chat/history/<conversation_id>`	JWT	
DELETE	`/api/chat/session/<session_id>`	JWT	

Method	Path	Auth	Rate limit
GET	`/api/portfolio/`	JWT	
GET	`/api/portfolio/quotes`	JWT	
POST	`/api/portfolio/holding`	JWT	
PUT	`/api/portfolio/holding/<holding_id>`	JWT	
DELETE	`/api/portfolio/holding/<holding_id>`	JWT	
GET	`/api/admin/users`	admin	
POST	`/api/admin/users/<user_id>/unlock`	admin	
GET	`/api/admin/audit-logs`	admin	

See Also

- SECURITY.md for auth, rate limiting and ownership rules
- BACKEND.md for where each route lives
- `backend/scripts/smoke_test.ps1` for a scripted walkthrough of these endpoints

7 Security

Purpose: How the app protects accounts and data, which controls exist and where they are in the code, and which known gaps remain.

Overview

Users authenticate with email and password, receive a short-lived JWT, and can only reach their own data. Logins and chat are rate limited, every request body is validated, and security-relevant events are written to an audit log that admins can read. The app is for financial *education* and stores no payment or bank data.

Authentication (JWT)

Setting	Value	Where
Library	Flask-JWT-Extended	`app/__init__.py`
Algorithm	HS256, signed with `JWT_SECRET_KEY`	`config.py`
Lifetime	1 hour (`JWT_ACCESS_TOKEN_EXPIRES`)	`config.py`
Subject	`str(user.id)`	`routes/auth.py`
Refresh tokens	none: users log in again	
Client storage	`localStorage["advisor_token"]`	`frontend/src/api.ts`

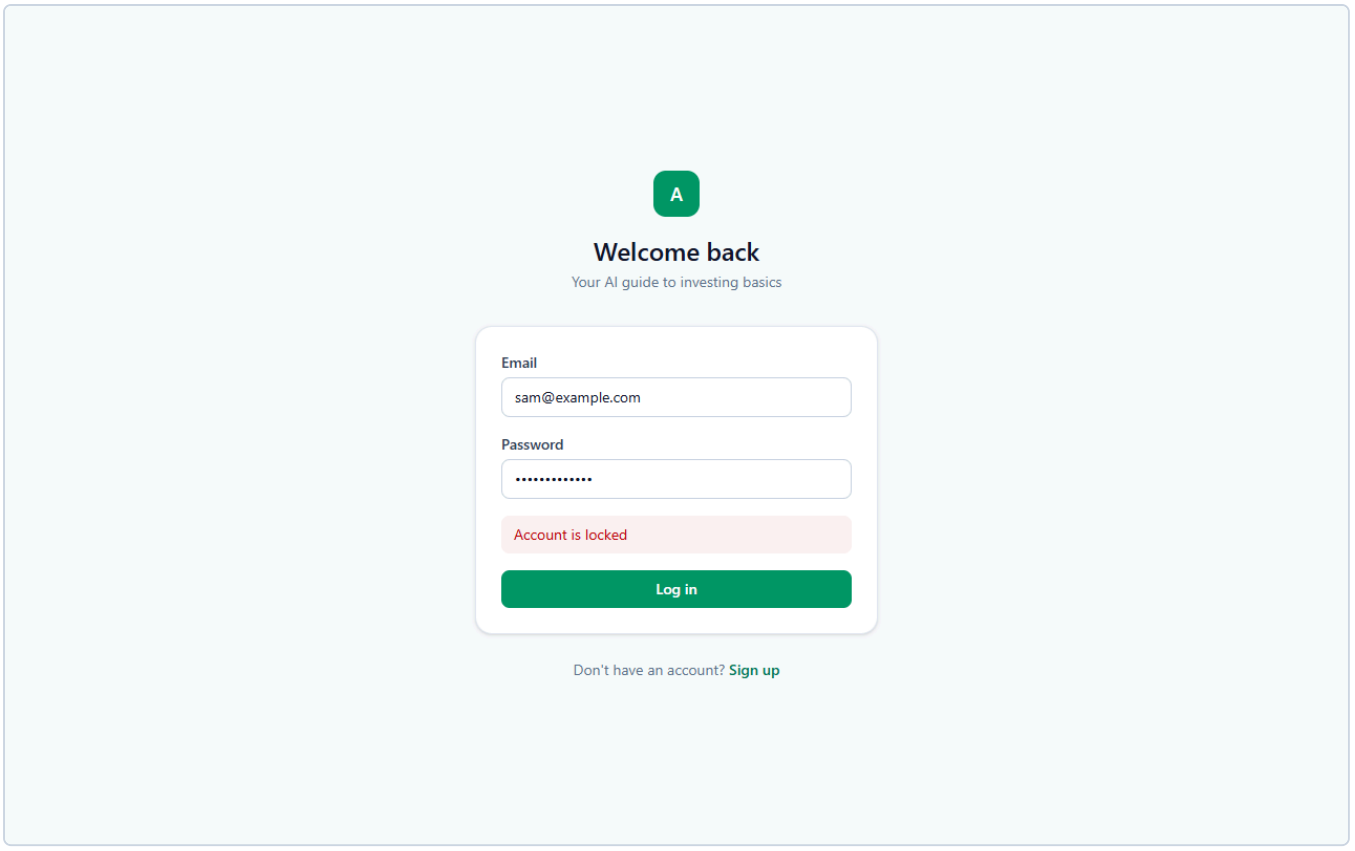
- Protected routes use `@jwt_required()` . Read the user with `current_user_id()` from `app/middleware` .
- Expired, malformed and missing tokens all return `401` with a JSON error. The frontend logs out on any 401 and shows "Your session expired. Please log in again."
- There is no server-side token revocation. To invalidate **every** token at once, change `JWT_SECRET_KEY` .
- In production `JWT_SECRET_KEY` must be set and at least 32 characters, or the app refuses to start (`ProductionConfig.validate()`).

Passwords

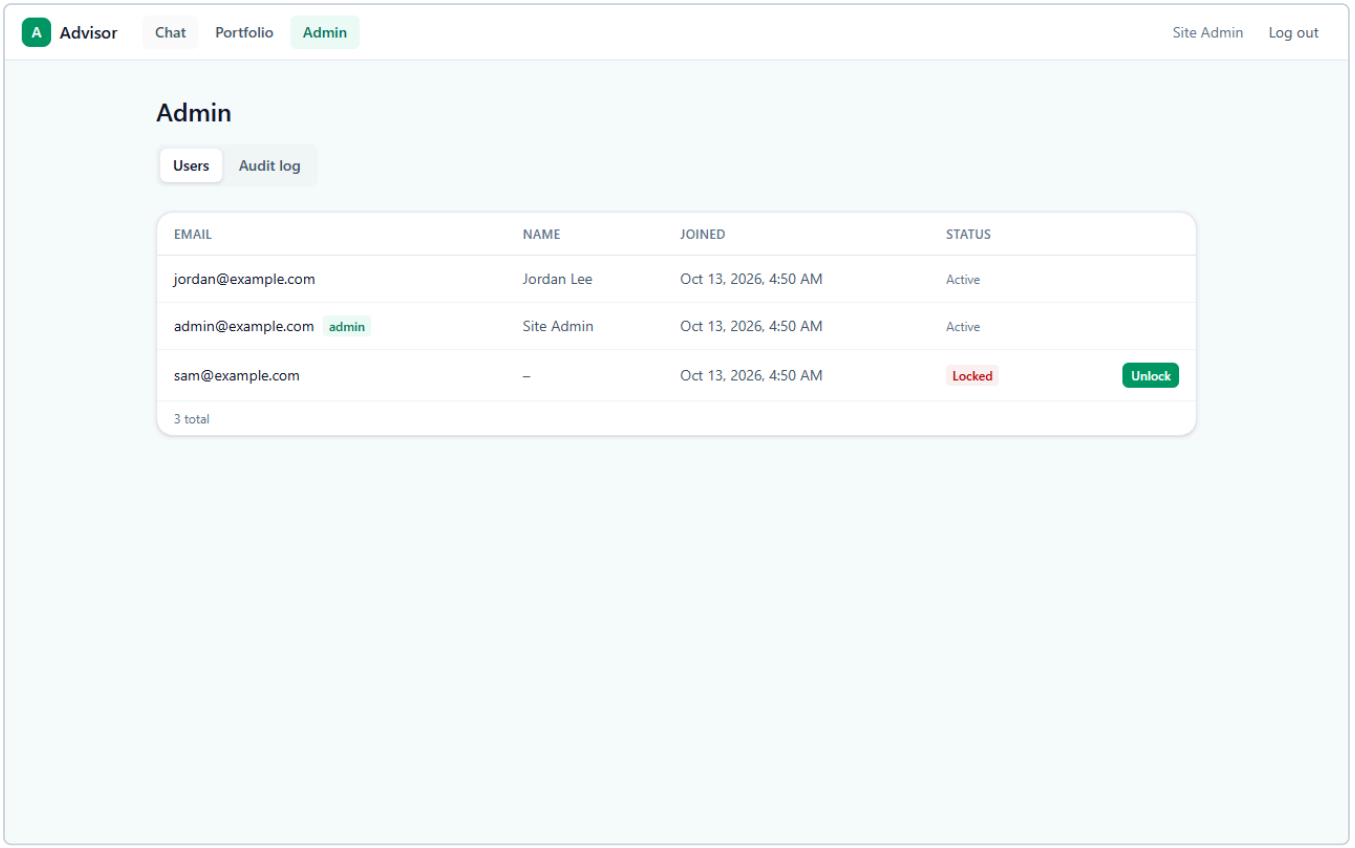
- Hashed with **bcrypt**, 12 rounds (`BCRYPT_ROUNDS` ; 4 in tests for speed).
- Length 8–128 characters, enforced by `RegisterSchema` .
- The hash is never returned: `User.to_dict()` leaves it out.
- Login returns the same message, `Invalid email or password` , whether the email is unknown or the password is wrong.

Account lockout

- Each wrong password increments `failed_login_attempts` . At `MAX_FAILED_LOGINS` (5) the account is locked.
- A successful login resets the counter.
- A locked account is refused with `403 Account is locked` , **even with the correct password**, and the attempt is logged as `login_blocked_locked` .
- Only an admin can unlock (Admin → Users → **Unlock**, or `POST /api/admin/users/<id>/unlock`), which also resets the counter.



Locked account at login



Admin users list with a locked account

Authorization and roles

Two roles, stored as `users.is_admin`:

Role	Can
User	Their own chats, messages and portfolio
Admin	Everything a user can, plus list users, unlock accounts, read the audit log

- Admin endpoints use `@admin_required`, which re-reads the user from the database on every request. Revoking admin takes effect immediately; it doesn't wait for the token to expire.
- Admin is granted only from the command line: `flask --app main make-admin <email>` (`--revoke` to remove). There is no API to grant it.
- The frontend hides the Admin tab and route for non-admins, but the backend check is what enforces it. A non-admin calling an admin endpoint gets `403 Admin access required`.

Ownership rule: every query for a user-owned resource filters by the JWT user, joining through parent tables:

```
# routes/chat.py
Conversation.query.join(ChatSession).filter(
    Conversation.id == conversation_id, ChatSession.user_id == user_id
)
```

Another user's resource returns **404, not 403**, so an attacker can't tell which IDs exist. Covered by `test_cannot_access_another_users_conversation`, `test_cannot_modify_another_users_holding` and `test_stream_rejects_other_users_conversation`.

The Gemini request for a user includes only that user's own conversation and holdings, so a prompt can't pull in another user's data.

Rate limiting

Flask-Limiter, storage from `RATELIMIT_STORAGE_URI`.

Endpoints	Default	Keyed by
`POST /auth/login`, `POST /auth/register`	`10 per minute`	client IP
`POST /chat/message` + `POST /chat/message/stream` (one shared allowance)	`10 per minute;200 per day`	user ID
`GET /health`	exempt	
everything else	none	

- Limits are configurable through `AUTH_RATE_LIMIT` and `CHAT_RATE_LIMIT` without code changes.
- Responses include `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`, and `Retry-After` on 429.
- With more than one gunicorn worker or instance, storage **must** be Redis; with `memory://` each process counts separately. Compose and Render use Redis.
- Behind a proxy, set `TRUSTED_PROXY_COUNT` to the number of proxies. Otherwise every request appears to come from the proxy's IP and all users share one login allowance. Set it too high and clients can spoof their IP with `X-Forwarded-For`.

The chat limit also caps Gemini spending per user.

Input validation

Every request body goes through a marshmallow schema in `app/schemas/__init__.py`; unknown fields are rejected.

Schema	Rules
`RegisterSchema`	valid email; password 8–128; name ≤ 120
`LoginSchema`	valid email; password required
`MessageSchema`	`conversation_id` int; `content` 1–4000 characters
`HoldingSchema`	symbol `^[A-Za-z.\-]{1,16}$`; quantity > 0; average cost ≥ 0 (decimals)
`CreateSessionSchema`	title ≤ 200

Failures return `400 {"error": "Invalid input", "details": {...}}`. URL IDs are typed (`<int:...>`) by the routes.

SQL injection

All database access goes through the SQLAlchemy ORM, which uses bound parameters. The code contains no raw SQL strings (no `text()` or `execute()` with string building).

XSS

- React escapes all text it renders.
- Advisor replies are rendered with `react-markdown` **without** `rehype-raw`, so HTML inside a reply is shown as text, not executed. Its default URL handling also drops `javascript:` links.
- No `dangerouslySetInnerHTML` anywhere in the frontend.
- Because the JWT is in `localStorage`, any XSS would be able to read it. That's the main reason to keep the points above true. See known gaps.

CORS

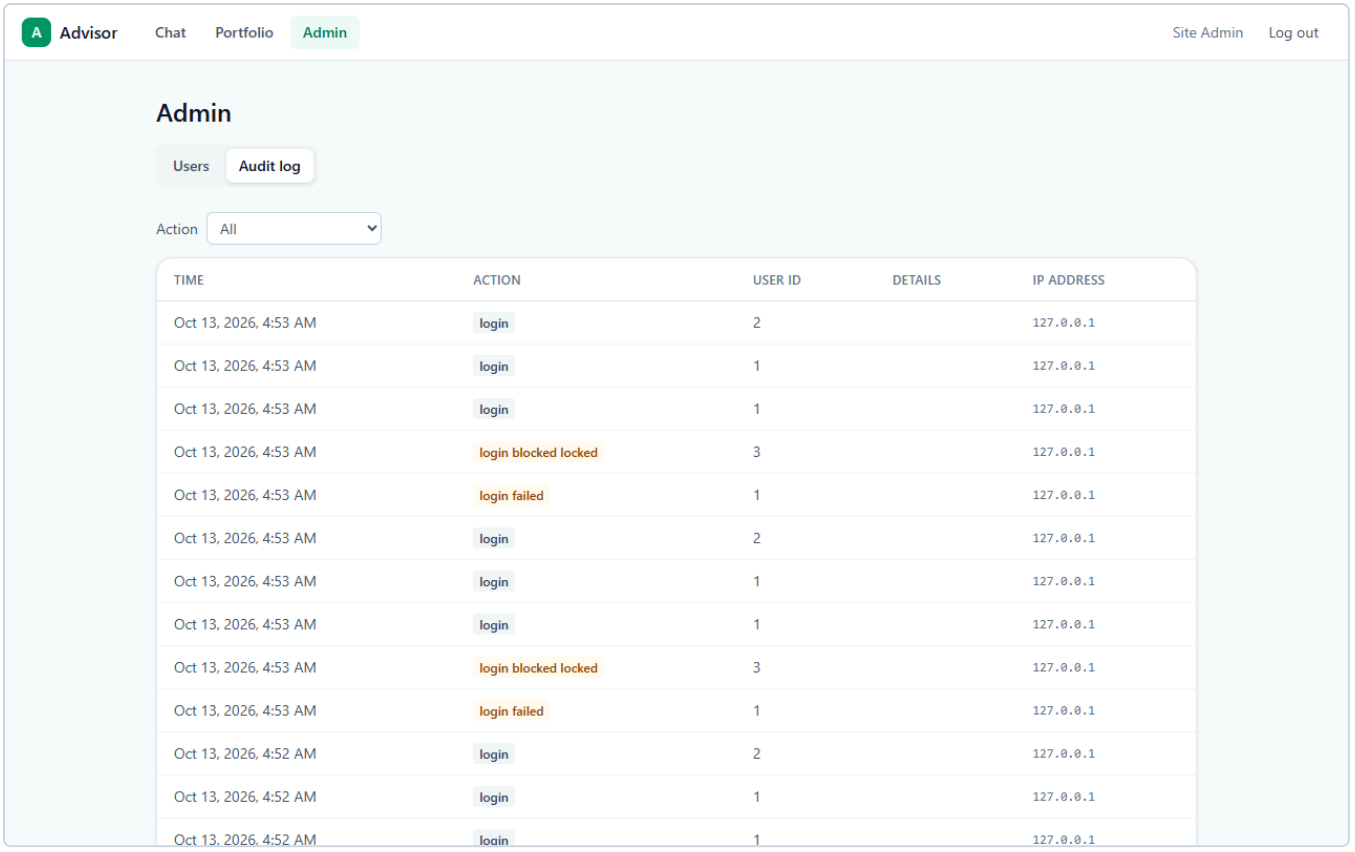
`CORS(app, origins=CORS_ORIGINS)`. Only the listed origins (comma-separated, exact match, no trailing slash) may call the API from a browser. Default `http://localhost:5173`. In development the Vite proxy makes requests same-origin, so CORS doesn't apply. In production set it to your Vercel URL.

Audit logging

`record_audit()` (`app/services/audit.py`) adds a row in the same transaction as the change it describes.

Action	When	`details`
`register`	Account created	
`login`	Successful login	
`login_failed`	Wrong password for an existing account	
`login_blocked_locked`	Login attempt on a locked account	
`account_unlocked`	Admin unlocked a user (`user_id` = the admin)	`user <id>`
`holding_added` / `holding_updated` / `holding_deleted`	Portfolio change	symbol

- Each row stores the client IP (real client IP when `TRUSTED_PROXY_COUNT` is set).
- `audit_logs` has no foreign key to `users`, so history survives account deletion.
- Failed logins for **unknown** emails are not logged (there's no user to attach them to); they are still rate limited.
- Chat messages are not audit-logged; they're stored as messages.



Audit log

Secrets and configuration

- Secrets live in `backend/.env` locally (git-ignored) and in the host's environment settings in production. Never commit them.
- Render generates `JWT_SECRET_KEY` and `SECRET_KEY` (`generateValue: true` in `render.yaml`).
- Generate your own with `python -c "import secrets; print(secrets.token_hex(32))"`.
- The development server binds to `127.0.0.1` only, because the Werkzeug debugger allows code execution.
- The backend Docker image runs as a non-root user (`appuser`).

Data privacy

Data	Stored where	Sent to
Email, name, password hash	App database	nobody
Chat messages	App database	**Google Gemini** (last 20 messages of the conversation, with each request)
Holdings (symbol, quantity, cost)	App database	**Google Gemini** (in the system prompt); **Yahoo Finance** receives symbols only

Data	Stored where	Sent to
IP addresses	`audit_logs`	nobody

- Users can delete their own conversations (deleting a session removes its messages).
- There is no self-service account deletion or data export yet.
- Gemini's data handling depends on your Google API terms; free-tier keys may allow Google to use prompts to improve its products. Use a paid key for real user data.
- Every chat screen states that answers are for education, not financial advice, and the system prompt tells the model the same.

Known gaps and recommendations

Gap	Risk	Suggested fix
JWT in `localStorage`	Stolen by any XSS	Move to an `HttpOnly`, `Secure`, `SameSite` cookie (Flask-JWT-Extended supports this, with CSRF protection)
No token revocation	A stolen token works for up to 1 hour	Add a token blocklist, or shorten the lifetime and add refresh tokens
No security headers	Clickjacking, weaker XSS defence	Add `Content-Security-Policy`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy` in `nginx.conf` / `vercel.json`
Lockout can be triggered by anyone who knows an email	Denial of service against one account	Time-based unlock, or lock per IP + account; CAPTCHA after failures
Email enumeration	`409 Email is already registered` and `403 Account is locked` reveal that an email exists	Generic responses; email verification flow
No email verification or password reset	Typos lock users out; admins can't reset passwords	Add a verified-email and reset flow
No password strength check beyond length	Weak passwords	Check against a breached-password list
Docker Compose uses Postgres password `advisor`	Weak if the port is ever exposed	Change it in `docker-compose.yml` (the port is not published by default)

See Also

- ARCHITECTURE.md for the login sequence
- TESTING.md for the security-related tests
- DEPLOYMENT.md for production secrets

8 Testing

Purpose: How to run the tests, what they cover, and how external services are faked.

Overview

There are two automated suites: **37 backend tests** (pytest) and **16 frontend tests** (Vitest), plus a lint step and a PowerShell smoke test that runs against a live server. CI runs the automated suites on every push and pull request. Neither suite calls Gemini or Yahoo Finance.

Running the tests

Backend, from `backend/` with the venv active:

```
pytest                                                  # all
pytest -v                                               # list each test
pytest tests/test_chat.py                               # one file
pytest tests/test_chat.py::test_history_is_passed_to_advisor  # one test
pytest -k lock                                          # by name match
```

Frontend, from `frontend/`:

```
npm test                       # vitest run
npx vitest run -t "streamed"   # tests whose name matches
npx vitest                     # watch mode
npm run lint                   # oxlint
npm run build                  # also type-checks the tests (tsc -b)
```

Latest results

Captured 2026-10-13 on the development machine.

Backend: **37 passed in 2.46s**

```
PS backend> pytest -v

rootdir: C:\Users\admin\Documents\financial-advisor-chatbot\backend
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1, platformdirs-4.12.4
collecting ... collected 37 items

tests/test_admin.py::test_non_admin_is_forbidden PASSED                  [  2%]
tests/test_admin.py::test_admin_lists_users_and_audit_logs PASSED        [  5%]
tests/test_admin.py::test_admin_unlocks_account PASSED                   [  8%]
tests/test_advisor.py::test_overloaded_then_rate_limited_then_success PASSED [ 10%]
tests/test_advisor.py::test_gives_up_with_a_friendly_message PASSED      [ 13%]
tests/test_advisor.py::test_non_retryable_errors_fail_immediately PASSED [ 16%]
tests/test_advisor.py::test_stream_retries_before_any_text PASSED        [ 18%]
tests/test_advisor.py::test_stream_does_not_retry_after_text_was_sent PASSED [ 21%]
tests/test_auth.py::test_register_and_login PASSED                       [ 24%]
tests/test_auth.py::test_register_duplicate_email PASSED                 [ 27%]
tests/test_auth.py::test_register_validates_input PASSED                 [ 29%]
tests/test_auth.py::test_account_locks_after_repeated_failures PASSED    [ 32%]
tests/test_auth.py::test_protected_route_requires_token PASSED           [ 35%]
tests/test_auth.py::test_health PASSED                                   [ 37%]
tests/test_chat.py::test_send_message_stores_both_sides PASSED           [ 40%]
tests/test_chat.py::test_untitled_session_is_named_after_first_message PASSED [ 43%]
tests/test_chat.py::test_history_is_passed_to_advisor PASSED             [ 45%]
tests/test_chat.py::test_cannot_access_another_users_conversation PASSED [ 48%]
tests/test_chat.py::test_advisor_failure_returns_503_and_saves_nothing PASSED [ 51%]
tests/test_cli.py::test_make_admin_command PASSED                        [ 54%]
tests/test_migrations.py::test_migrations_build_the_current_schema PASSED [ 56%]
tests/test_portfolio.py::test_add_update_delete_holding PASSED           [ 59%]
tests/test_portfolio.py::test_duplicate_symbol_conflicts PASSED          [ 62%]
tests/test_portfolio.py::test_cannot_modify_another_users_holding PASSED [ 64%]
tests/test_portfolio.py::test_portfolio_is_given_to_advisor PASSED       [ 67%]
tests/test_quotes.py::test_quotes_endpoint_returns_prices_and_tolerates_bad_symbols PASSED [ 70%]
tests/test_quotes.py::test_quotes_are_cached PASSED                      [ 72%]
tests/test_quotes.py::test_quotes_disabled_returns_nothing PASSED        [ 75%]
tests/test_quotes.py::test_prices_are_given_to_the_advisor PASSED        [ 78%]
tests/test_rate_limits.py::test_chat_is_rate_limited_per_user PASSED     [ 81%]
tests/test_rate_limits.py::test_login_is_rate_limited_per_ip PASSED      [ 83%]
tests/test_rate_limits.py::test_chat_limit_is_shared_between_streaming_and_plain_endpoints PASSED [ 86%]
tests/test_streaming.py::test_stream_sends_deltas_then_saves PASSED      [ 89%]
tests/test_streaming.py::test_stream_error_mid_reply_saves_nothing PASSED [ 91%]
tests/test_streaming.py::test_stream_rejects_other_users_conversation PASSED [ 94%]
tests/test_streaming.py::test_stream_request_includes_history_and_portfolio PASSED [ 97%]
tests/test_streaming.py::test_stream_with_deleted_conversation_saves_nothing PASSED [100%]

============================ 37 passed in 2.46s ============================
```

pytest output

Frontend: **16 passed** in 3 files

```
PS frontend> npm test

 RUN  v5.0.3 C:/Users/admin/Documents/financial-advisor-chatbot/frontend


 Test Files  3 passed (3)
      Tests  16 passed (16)
   Start at  12:54:38
   Duration  29.66s (environment 78%, setup 16%, tests 3%, import 3%, transform 1%)
```

vitest output

Lint (`npm run lint`): 0 errors, 3 warnings (`only-export-components` in `auth.tsx` , `set-state-in-effect` in `ChatPage.tsx` and `PortfolioPage.tsx`).

Backend tests

Each test gets a fresh app with `create_app("testing")` : in-memory SQLite, bcrypt with 4 rounds, quotes disabled, retry delays of zero, and in-memory rate limits.

File	Tests	Covers
`test_auth.py`	6	Register + login, duplicate email (409), validation (400), lockout after 5 failures, token required, health
`test_chat.py`	5	Both messages saved, session titled from first message, history passed to the advisor, other users' conversations return 404, Gemini failure → 503 with nothing saved

File	Tests	Covers
`test_streaming.py`	5	SSE deltas then `done` and save, mid-stream error saves nothing, ownership, request includes history and portfolio, conversation deleted mid-stream
`test_advisor.py`	5	Retry on 503 then 429 then success, friendly message after retries run out, non-retryable errors fail at once, streams retry only before any text
`test_portfolio.py`	4	Add / update / delete, duplicate symbol (409), other users' holdings return 404, holdings given to the advisor
`test_quotes.py`	4	Quotes endpoint tolerates bad symbols, caching, disabled quotes, prices included in the advisor prompt
`test_rate_limits.py`	3	Chat limited per user, login limited per IP, one allowance shared by both message endpoints
`test_admin.py`	3	Non-admin gets 403, admin lists users and logs, unlock
`test_cli.py`	1	`make-admin` grant and revoke
`test_migrations.py`	1	Running all migrations produces exactly the current models' schema

Shared fixtures are in `tests/conftest.py`: `app`, `client`, `register` (creates a user, returns `(user_id, auth headers)`, optionally admin), `fake_advisor`, `fake_stream` and `parse_sse`.

Frontend tests

Vitest with jsdom and Testing Library.

File	Tests	Covers
`src/App.test.tsx`	6	Login error shown; login lands on chat; new conversation shows streamed reply; failed send keeps the typed text and shows the error; portfolio value and gain; slow outdated quote response is ignored
`src/api.test.ts`	7	`parseSSE` (partial events, CRLF); `streamMessage` (split chunks, error event, JSON error before stream); 401 triggers logout; first validation detail as friendly message
`src/portfolio.test.ts`	3	Valuation with and without prices, sorting, empty portfolio

Mocking external services

Rule: tests never call Gemini or Yahoo.

Service	How it's faked
Gemini, plain	`fake_advisor` fixture replaces `app.routes.chat.generate_reply` with a canned reply and records the messages and portfolio it was given
Gemini, streaming	`fake_stream` fixture replaces `app.routes.chat.stream_reply`, yields two chunks, and records the request
Gemini SDK errors	`test_advisor.py` monkeypatches `advisor._get_client` with a fake client that raises `google.genai.errors` with chosen status codes
Yahoo Finance	`QUOTES_ENABLED = False` in tests; quote tests turn it on and monkeypatch `quotes._fetch_quote`
Backend (from the frontend)	`src/test/fakeServer.ts` replaces `fetch` with routes keyed by `"METHOD /path"`. Unmatched requests throw, so unexpected calls fail the test

`fake_stream` calls `close_request_session()` before yielding. A real server closes the request's DB session before the streamed body runs, but Flask's test client doesn't, so without this a test could pass while production fails.

Frontend example:

```
fakeServer({
  'GET /auth/me': () => json({ user }),
  'POST /chat/session': () => json({ session, conversation_id: 7 }, 201),
  'POST /chat/message/stream': () =>
    eventStream([sse({ type: 'delta', text: 'Hello' }), sse({ type: 'done', user_message,
assistant_message })]),
})
```

Smoke test against a live server

`backend/scripts/smoke_test.ps1` exercises a running server end to end, **including one real Gemini call**. It checks: server is up, register, login, add holdings, chat with Gemini, chat history is saved, missing token (401), non-admin (403) and wrong password (401). Each check prints PASS or FAIL.

```
cd backend
.\scripts\smoke_test.ps1                                    # dev server
.\scripts\smoke_test.ps1 -BaseUrl http://localhost:8080/api  # Docker Compose
```

It creates a new random user each run, so run it against development or staging data only.

CI pipeline

`.github/workflows/ci.yml`, on push to `main` and on pull requests:

Job	Steps
`backend`	Python 3.13 → `pip install -r requirements.txt` → `pytest`
`frontend`	Node 24 → `npm ci` → `npm run lint` → `npm test` → `npm run build`
`docker`	`docker build backend` and `docker build frontend`

Coverage and load testing

- **Coverage:** not measured yet. To add it: `pip install pytest-cov` then `pytest --cov=app --cov-report=term-missing`; for the frontend, `npm i -D @vitest/coverage-v8` then `npx vitest run --coverage`. A reasonable first target is 80% of backend lines, with routes and `advisor.py` fully covered.
- **Load testing:** not set up. Chat throughput is bounded by the Gemini quota and by `CHAT_RATE_LIMIT`, so a load test should fake Gemini and focus on gunicorn worker and thread counts (`WEB_CONCURRENCY`, `GUNICORN_THREADS`) for concurrent streams.

Writing a new test

```
# backend/tests/test_portfolio.py
def test_symbol_is_uppercased(client, register):
    _, headers = register()
    response = client.post(
        "/api/portfolio/holding",
        json={"symbol": "vti", "quantity": "1", "average_cost": "200"},
        headers=headers,
    )
    assert response.status_code == 201
    assert response.get_json()["holding"]["symbol"] == "VTI"
```

Anything that would reach Gemini needs `fake_advisor` or `fake_stream` in the test's arguments.

See Also

- BACKEND.md for the conventions the tests enforce
- TROUBLESHOOTING.md if a test fails locally

9 Deployment

Purpose: Take the app from GitHub to a live URL, and operate it once it's there.

The production setup is three free-tier services:

Part	Host	Config in this repo
Database (Postgres)	Supabase	none, just a connection string
Backend API + Redis	Render	`render.yaml`
Frontend	Vercel	`frontend/vercel.json`

Do them in this order, because each step needs a URL from the one before.

Alternatively, run everything on any server with Docker using `docker compose up -d --build` (see the end of this guide).

0. Push the code to GitHub

Render and Vercel deploy from a GitHub repository.

```
git remote add origin https://github.com/<you>/financial-advisor-chatbot.git
git push -u origin main
```

The `CI` workflow (`.github/workflows/ci.yml`) then runs the backend tests, frontend lint/tests/build and both Docker builds on every push.

1. Database: Supabase

1. Create a project and save the database password you choose.
2. Click **Connect** at the top of the project page, then copy the **Session pooler** connection string. Render can't reach Supabase's direct connection, which is IPv6-only. It looks like: `postgresql://postgres.<project-ref>:<password>@aws-0-<region>.pooler.supabase.com:5432/postgres`
3. Replace `[YOUR-PASSWORD]` with your password and add `?sslmode=require` to the end.

You don't need to create any tables. The backend runs its migrations every time it starts.

2. Backend: Render

1. Render dashboard → **New** → **Blueprint** → choose the repository. Render reads `render.yaml` and proposes the `advisor-api` web service and the `advisor-ratelimit` Redis.
2. Fill in the values it asks for:
   - `DATABASE_URL` : the Supabase string from step 1
   - `GEMINI_API_KEY` : from https://aistudio.google.com/apikey (consider a separate key from your development one)
   - `CORS_ORIGINS` : leave as `https://example.com` for now and fix it in step 4

   `JWT_SECRET_KEY` and `SECRET_KEY` are generated for you.
3. Deploy. When it's live, open `https://advisor-api-XXXX.onrender.com/api/health` and check that it returns `{"status": "ok"}` .

Free Render services sleep after 15 minutes without traffic, so the first request afterwards takes about a minute.

3. Frontend: Vercel

1. Vercel dashboard → **Add New** → **Project** → import the repository.
2. Set **Root Directory** to `frontend`. Vercel detects Vite and fills in the build settings.
3. Add the environment variable `VITE_API_URL` = `https://advisor-api-XXXX.onrender.com/api` (your Render URL plus `/api`).
4. Deploy and note the URL, e.g. `https://advisor-abc.vercel.app`.

`VITE_API_URL` is baked in when the site is built. If you change it, redeploy.

4. Connect them

In Render → `advisor-api` → **Environment**, set `CORS_ORIGINS` to your exact Vercel URL, with no trailing slash. You can list several, separated by commas. Save; Render redeploys.

Then open the Vercel URL, sign up, add a holding and ask a question.

5. Make yourself an admin

Sign up on the live site first. Render's free plan has no Shell, so run the command from your own machine against the production database (from `backend/`, venv active). In PowerShell:

```
$env:DATABASE_URL = "<your Supabase connection string>"
flask --app main make-admin you@example.com
Remove-Item Env:DATABASE_URL
```

On a paid Render plan you can instead run `flask --app main make-admin you@example.com` in **advisor-api** → **Shell**. Log out and back in to see the **Admin** tab.

Operating notes

- **Schema changes:** after changing `app/models`, run `flask --app main db migrate -m "describe change"` locally, review the generated file in `backend/migrations/versions/`, and commit it. Deploys apply it automatically.
- **Rate limits:** adjust `CHAT_RATE_LIMIT` / `AUTH_RATE_LIMIT` in Render's environment (e.g. `20 per minute;500 per day`).
- **Live prices** come from Yahoo Finance's unofficial API, which is fine for personal use. For a commercial product, replace `_fetch_quote` in `backend/app/services/quotes.py` with a licensed data provider, or set `QUOTES_ENABLED=false`.
- **Secrets** live only in the hosts' environment settings, never in git. To revoke every login at once, change `JWT_SECRET_KEY`.

Environment variables checklist

Backend (Render → `advisor-api` → Environment, or `backend/.env` for Docker):

Variable	Production value	Set by
`FLASK_ENV`	`production`	`render.yaml`
`DATABASE_URL`	Supabase session pooler URL + `?sslmode=require`	you
`GEMINI_API_KEY`	your key	you
`CORS_ORIGINS`	exact Vercel URL, no trailing slash	you (step 4)
`JWT_SECRET_KEY`	32+ random characters	generated by Render
`SECRET_KEY`	random	generated by Render
`RATELIMIT_STORAGE_URI`	Redis connection string	`render.yaml` (from `advisor-ratelimit`)
`TRUSTED_PROXY_COUNT`	`1`	`render.yaml`
`GEMINI_MODEL`, `CHAT_RATE_LIMIT`, `AUTH_RATE_LIMIT`, `QUOTES_ENABLED`	optional, defaults are fine	you
`WEB_CONCURRENCY`, `GUNICORN_THREADS`	optional; default 2 workers × 8 threads	you

Frontend (Vercel → Project → Settings → Environment Variables):

Variable	Value
`VITE_API_URL`	`https://advisor-api-XXXX.onrender.com/api`

The backend refuses to start in production without `JWT_SECRET_KEY` (32+ characters), `SECRET_KEY` and `DATABASE_URL`.

CI/CD

- **CI:** `.github/workflows/ci.yml` runs on every push to `main` and every pull request: backend `pytest`, frontend lint + tests + build, and both Docker builds. See TESTING.md.
- **CD:** Render and Vercel both auto-deploy when `main` changes. They don't wait for CI, so check the Actions tab before merging. To gate deploys on CI, turn off auto-deploy in Render and use a deploy hook from a workflow step after the tests pass.
- **Migrations** run on every backend start (`docker-entrypoint.sh` → `flask --app main db upgrade`), before gunicorn takes traffic.
- **Rollback:** Render → `advisor-api` → Events → pick an earlier deploy → **Rollback**; Vercel → Deployments → **Promote to Production**. A rollback doesn't undo a migration, so keep migrations backward compatible.

Monitoring and logging

What exists today:

- **Health check:** `GET /api/health` → `{"status": "ok"}`. Render uses it (`healthCheckPath`), and so does the Docker `HEALTHCHECK`. It confirms the process is serving, not that the database or Gemini is reachable.
- **Logs:** gunicorn writes access logs and the app writes warnings and errors to stdout. View them in Render → Logs, or `docker compose logs -f backend`. Useful messages to watch for:
  - `Gemini returned 429; retrying in 1s`: you're near your Gemini quota.
  - `Gemini reply hit GEMINI_MAX_OUTPUT_TOKENS and was cut off`: raise the limit.

    - `Unhandled error` / `Error while streaming a reply`, followed by a traceback: a bug.
- **Audit log:** sign-ups, logins, failed and blocked logins, unlocks and portfolio changes, in the app's **Admin** → **Audit log** tab.
- **Usage:** Gemini usage and quota in Google AI Studio; database size in Supabase.

Not set up yet (recommended before real users): uptime monitoring on `/api/health` (e.g. UptimeRobot, which also keeps the free Render service awake), error tracking (e.g. Sentry's Flask and React SDKs), and Supabase backups (daily backups need a paid plan; otherwise schedule a `pg_dump`).

Alternative: one server with Docker Compose

On any Linux server with Docker:

```
git clone <repo> && cd financial-advisor-chatbot
cp backend/.env.example backend/.env   # set GEMINI_API_KEY, JWT_SECRET_KEY, SECRET_KEY
docker compose up -d --build
```

The app is served on port 8080. The bundled Postgres stores its data in the `pgdata` Docker volume; back it up. For HTTPS, put a reverse proxy such as Caddy in front, and set `TRUSTED_PROXY_COUNT=2` in `docker-compose.yml`, since there are then two proxies in front of the backend. Also change the Postgres password in `docker-compose.yml`.

See Also

- SETUP.md to run the same stack locally
- SECURITY.md for handling secrets
- TROUBLESHOOTING.md for production errors

10 Troubleshooting

Purpose: Find the cause and fix for common errors, from local setup to production.

Overview

Find the message you're seeing (in the browser, terminal or API response) in the sections below. Most problems are a server that isn't running, a missing environment variable, or a proxy or CORS setting. Backend logs are in the terminal running `python main.py`, in `docker compose logs backend`, or in Render's **Logs** tab.

Backend won't start

Error	Cause	Fix
`ModuleNotFoundError: No module named 'flask'`	venv not active	`venv\Scripts\Activate.ps1` (PowerShell) or `source venv/Scripts/activate` (Git Bash), then `pip install -r requirements.txt`
`running scripts is disabled on this system` when activating	PowerShell execution policy	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`RuntimeError: Missing required environment variables for production: JWT_SECRET_KEY, ...`	`FLASK_ENV=production` without secrets	Set `JWT_SECRET_KEY`, `SECRET_KEY`, `DATABASE_URL`, or use `FLASK_ENV=development` locally
`RuntimeError: JWT_SECRET_KEY must be at least 32 characters in production`	Short key	`python -c "import secrets; print(secrets.token_hex(32))"`
Server stopped after you saved a file	The auto-reloader hit a syntax error	Fix the error, then run `python main.py` again

Port already in use

`OSError: [WinError 10048]` or `Address already in use` for port 5000 (backend) or 5173 (frontend). Usually an earlier server is still running.

Windows (PowerShell):

```
netstat -ano | findstr :5000          # last column is the PID
taskkill /PID <pid> /F
```

macOS / Linux:

```
lsof -i :5000
kill <pid>
```

Or run on another port: `$env:PORT = "5001"; python main.py`, then start the frontend with `$env:API_PROXY_TARGET = "http://127.0.0.1:5001"; npm run dev`.

Frontend can't reach the backend

Symptom	Fix
Red banner **"Cannot reach the server. Is the backend running?"**	Start the backend; check http://127.0.0.1:5000/api/health returns `{"status": "ok"}`
Vite terminal shows `http proxy error: /api/... ECONNREFUSED`	Same: the backend isn't on the proxy target. Check `API_PROXY_TARGET`
Page loads but every call returns 404 HTML	You opened the backend port (5000) instead of the frontend (5173)

Database errors

Error	Cause	Fix
`sqlite3.OperationalError: no such table: users`	Migrations not applied	`python main.py` applies them on start, or `flask --app main db upgrade`
`test_migrations_build_the_current_schema` fails	Model changed without a migration	`flask --app main db migrate -m "describe change"`, review, commit
Want a clean dev database		Stop the server, delete `backend/instance/dev.db`, start it again
`connection to server ... failed: Network is unreachable` (Render → Supabase)	Supabase's direct connection is IPv6-only	Use the **Session pooler** connection string
`SSL connection is required`		Add `?sslmode=require` to `DATABASE_URL`
`password authentication failed`	`[YOUR-PASSWORD]` placeholder still in the URL, or special characters not URL-encoded	Fix the password; URL-encode `@ : / ?` etc.

`postgres://` URLs are fine; the app rewrites them to the psycopg 3 driver.

Advisor (Gemini) errors

These appear in red above the chat input. Your message goes back into the input box and nothing is saved, so you can resend.

Message	Cause	Fix
`GEMINI_API_KEY is not configured`	No key in the backend's environment	Set it in `backend/.env` (or Render), restart
`The advisor is getting too many requests right now. Please wait a minute and try again.`	Gemini returned 429 three times (after 1s and 3s retries). Common on free keys	Wait; lower `CHAT_RATE_LIMIT`; use a paid key
`The advisor is temporarily overloaded. Please try again in a moment.`	Gemini returned 5xx three times	Wait and retry; Google-side

Message	Cause	Fix
`The advisor service is unavailable`	Non-retryable error, usually an invalid or revoked key, or a wrong `GEMINI_MODEL`	Check the backend log for the Gemini error; fix the key or model name
`The advisor returned an empty response`	Gemini returned no text (e.g. safety block)	Rephrase the question
Answer ends with *"(This answer was cut off because it hit the length limit.)"*	Hit `GEMINI_MAX_OUTPUT_TOKENS`; 2.5 models spend part of it on thinking	Raise `GEMINI_MAX_OUTPUT_TOKENS` (default 8192)
`The connection closed before the reply finished`	Stream cut off: proxy timeout, server restart, network drop	Resend. Behind nginx keep `proxy_buffering off` and `proxy_read_timeout 120s`
`This conversation was deleted`	Chat was deleted in another tab while the reply streamed	Start a new chat

Login and session problems

Symptom	Cause	Fix
`Invalid email or password`	Wrong email or password	Check both; after 5 wrong passwords the account locks
`Account is locked`	5 failed logins in a row	An admin unlocks it in **Admin** → **Users** → **Unlock**. Correct passwords don't work until then
"Your session expired. Please log in again."	Tokens last 1 hour	Log in again
Everyone was logged out at once	`JWT_SECRET_KEY` changed (or wasn't set and differs between restarts/instances)	Expected after rotation; otherwise set a fixed key
No **Admin** tab after `make-admin`	The page still has the old user details	Log out and back in
`No user with email ...` from `make-admin`	User hasn't registered, or you're pointing at another database	Register first; check `DATABASE_URL`

The only admin is locked out: from `backend/` with the venv active (and `DATABASE_URL` pointing at the right database):

```
flask --app main shell
>>> from app import db
>>> from app.models import User
>>> u = User.query.filter_by(email="admin@example.com").first()
>>> u.account_locked = False; u.failed_login_attempts = 0
>>> db.session.commit()
```

Too many requests (429)

`Too many requests (10 per 1 minute). Please wait a moment and try again.`

- **Login / register:** 10 per minute per IP. Wait for the time in the `Retry-After` header.
- **Chat:** 10 per minute and 200 per day per user, shared by both message endpoints.

- **Everyone hits the login limit at once in production:** all requests look like they come from the proxy. Set `TRUSTED_PROXY_COUNT` to the number of proxies (1 on Render or Compose, 2 with an extra proxy such as Caddy in front).
- **Limits seem higher than configured with several workers:** `RATELIMIT_STORAGE_URI` is `memory://`; use Redis.
- Change limits with `CHAT_RATE_LIMIT` and `AUTH_RATE_LIMIT`, e.g. `20 per minute;500 per day`.

Portfolio prices

Symptom	Cause	Fix
Price column shows `n/a` for one holding	Yahoo doesn't know the symbol, or the fetch failed	Check the ticker on finance.yahoo.com; non-US tickers need a suffix (e.g. `SHOP.TO`)
"Could not load live prices right now; values use the prices you paid."	Quotes request failed	Click **Refresh prices**; check the backend's internet access
"Live prices are turned off, so values use the prices you paid."	`QUOTES_ENABLED=false`	Set it to `true` and restart
Price doesn't change after Refresh	Prices are cached for 60 seconds per server process, and Yahoo itself may be delayed	Wait a minute
Symbol rejected	Only letters, `.` and `-`, up to 16 characters	Use the plain ticker
`VTI is already in the portfolio; update it instead`	One row per symbol	Use **Edit** on the existing row

Docker

Error	Cause	Fix
`failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine`	Docker Desktop isn't running	Start Docker Desktop and wait until it says running
`backend` exits right away; log shows `Missing required environment variables`	`backend/.env` lacks `JWT_SECRET_KEY` / `SECRET_KEY` (Compose runs in production mode)	Set both, `docker compose up --build`
`frontend` waits forever	Backend health check failing	`docker compose logs backend`
Chat reply appears all at once instead of streaming	A proxy is buffering	nginx needs `proxy_buffering off` (already in `frontend/nginx.conf`); the backend also sends `X-Accel-Buffering: no`
Code changes don't show up	Images are built once	`docker compose up --build`
Start from an empty database		`docker compose down -v` (**deletes all data** in the `pgdata` volume)

Production (Render / Vercel / Supabase)

Symptom	Cause	Fix
Browser console: `blocked by CORS policy`	`CORS_ORIGINS` doesn't exactly match the Vercel URL	Set it to the exact origin, `https://...vercel.app`, no trailing slash; Render redeploys
Frontend calls `/api/...` on vercel.app and gets 404 or HTML	`VITE_API_URL` not set at build time	Set it in Vercel to `https://<render-url>/api` and **redeploy**
First request takes about a minute	Free Render services sleep after 15 minutes idle	Expected on the free plan
Deep link like `/chat/12` returns 404 on Vercel	Missing SPA rewrite	`frontend/vercel.json` must be deployed (Root Directory = `frontend`)
`/api/health` fails on Render	Startup error	Render → Logs; usually `DATABASE_URL` or missing secrets

Tests

Symptom	Fix
A backend test tries to reach Gemini	Add the `fake_advisor` or `fake_stream` fixture to it
Frontend test fails with `Unexpected request: GET /something`	Add that route to the test's `fakeServer({...})`
`npm run build` fails but `npm test` passes	The build type-checks test files too; fix the TypeScript error it reports

Known issues

- **Times in the admin panel are off by your UTC offset in development.** SQLite doesn't store the timezone, so the API returns timestamps like `2026-10-13T04:53:56` without `+00:00`, and the browser reads them as local time. Postgres (Docker and production) returns `+00:00` and displays correctly. A fix would be to add the UTC offset when serialising naive datetimes in the models' `to_dict()`.

See Also

- SETUP.md for setup steps
- DEPLOYMENT.md for production configuration
- SECURITY.md for lockout and rate-limit behaviour

11 User Guide

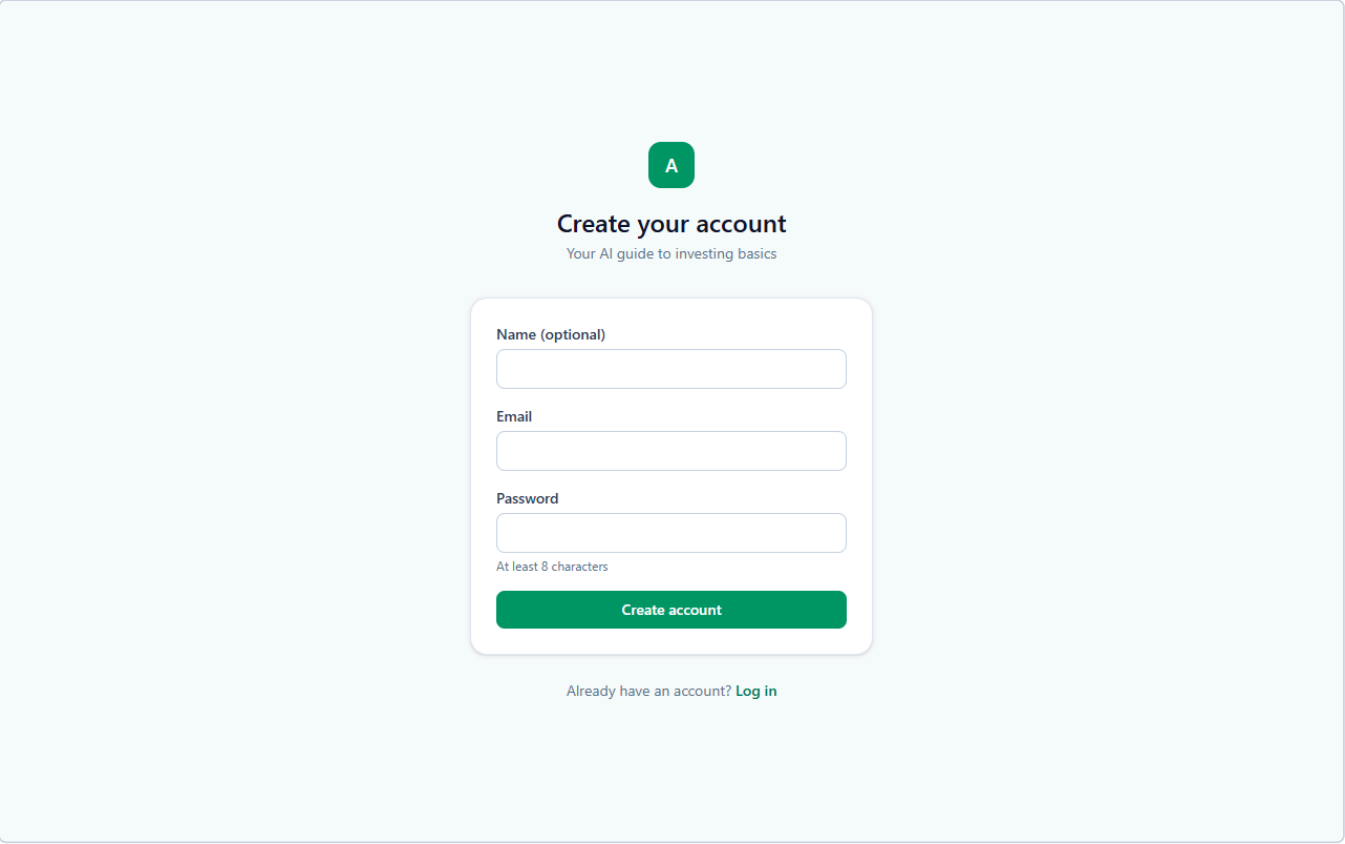
Purpose: How to use Advisor, the AI investing guide, step by step.

Overview

Advisor is a chat assistant that explains investing in plain language. If you enter the stocks and funds you own, it takes them, and their current prices, into account when it answers. It's an educational tool: it does not give personal financial advice, and you should talk to a licensed professional before making big decisions.

Create an account

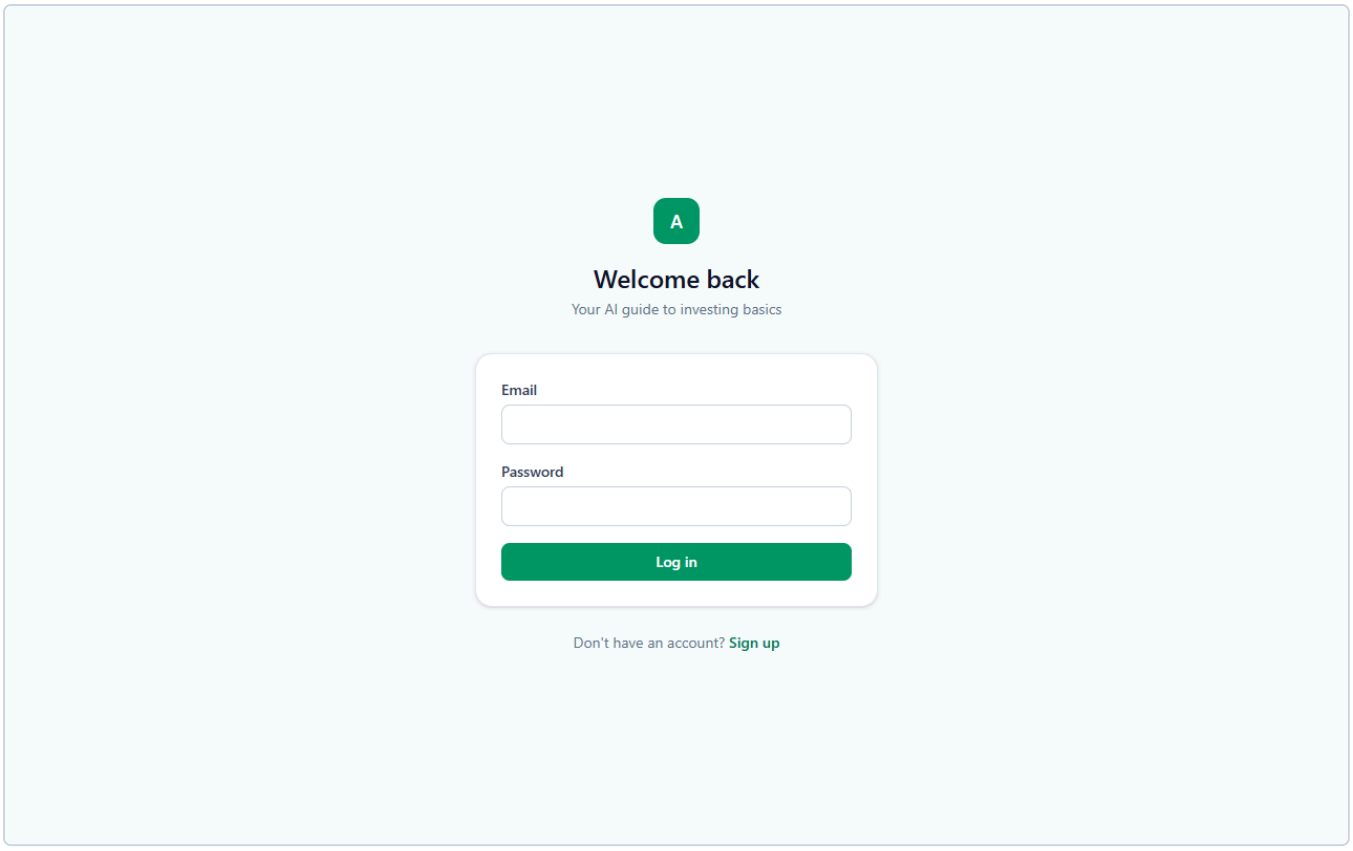
1. Open the app and click **Sign up** under the login form.
2. Enter your email and a password of at least 8 characters. Your name is optional; it's shown in the top bar.
3. Click **Create account**. You're logged in straight away.



Create account page

Log in

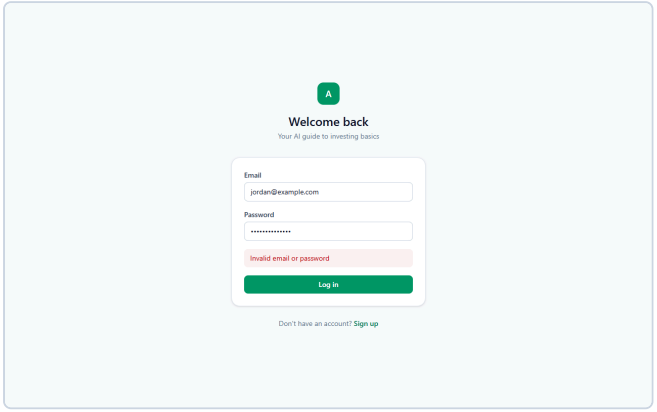
Enter your email and password and click **Log in**.



Login page

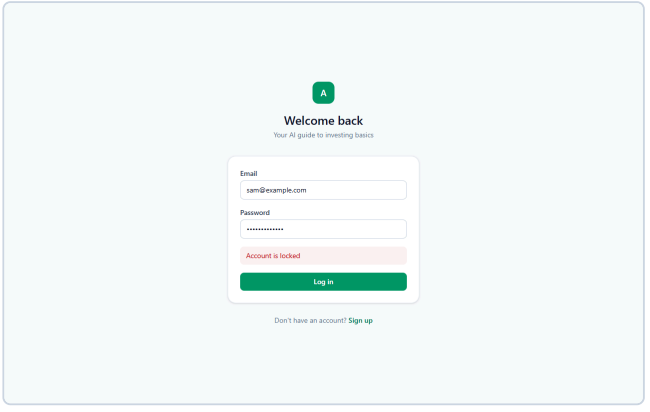
- If you see **Invalid email or password**, check both and try again.
- After **5 wrong passwords in a row** your account is locked for safety and you'll see **Account is locked**. Contact an administrator to unlock it.
- You stay logged in for up to an hour. After that you'll see "Your session expired. Please log in again."

Wrong password



Wrong password

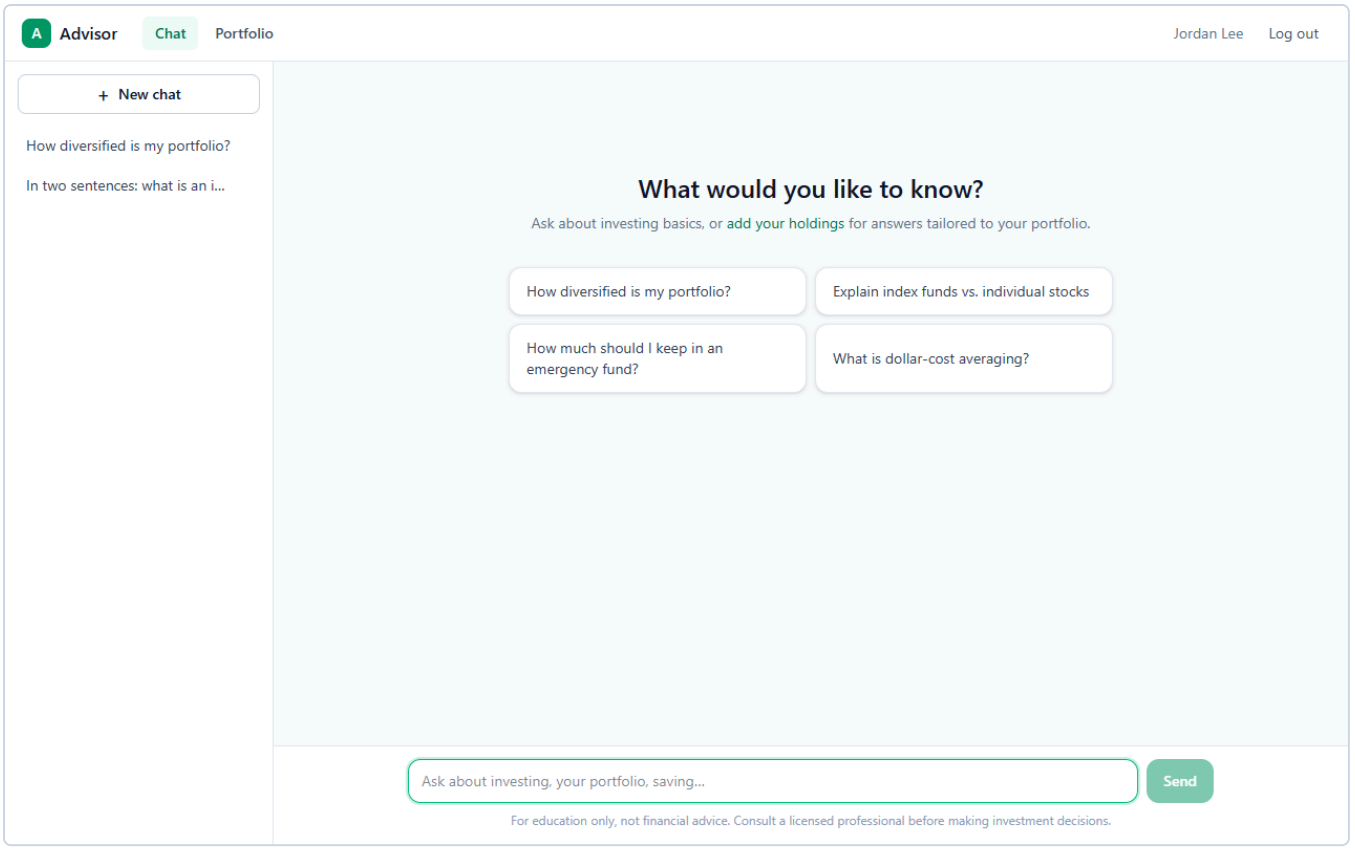
Locked account



Locked account

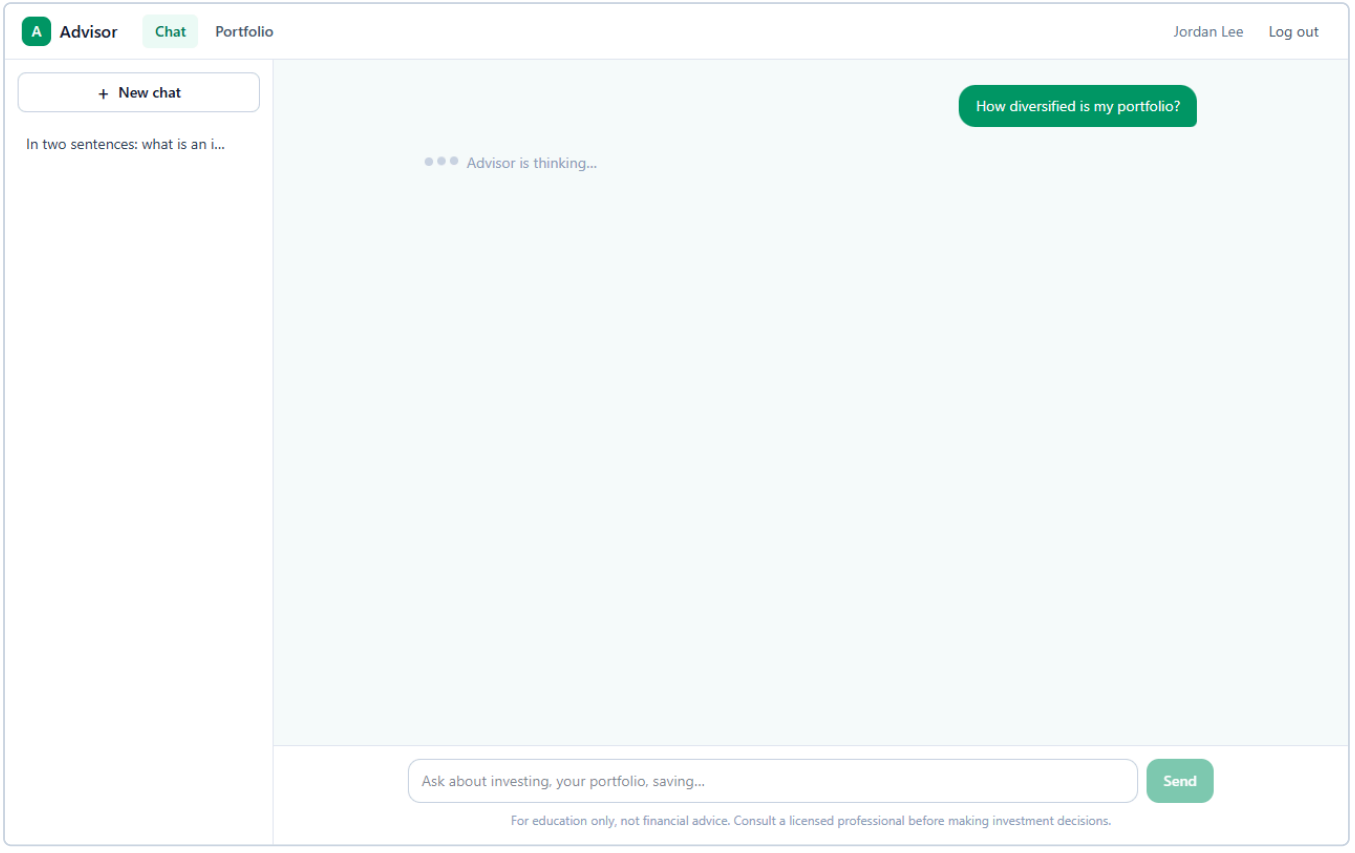
Chat with the advisor

1. Click **Chat** in the top bar.
2. Type a question in the box at the bottom, or click one of the suggestions to start.
3. Press **Enter** or click **Send**. Use **Shift+Enter** for a new line.

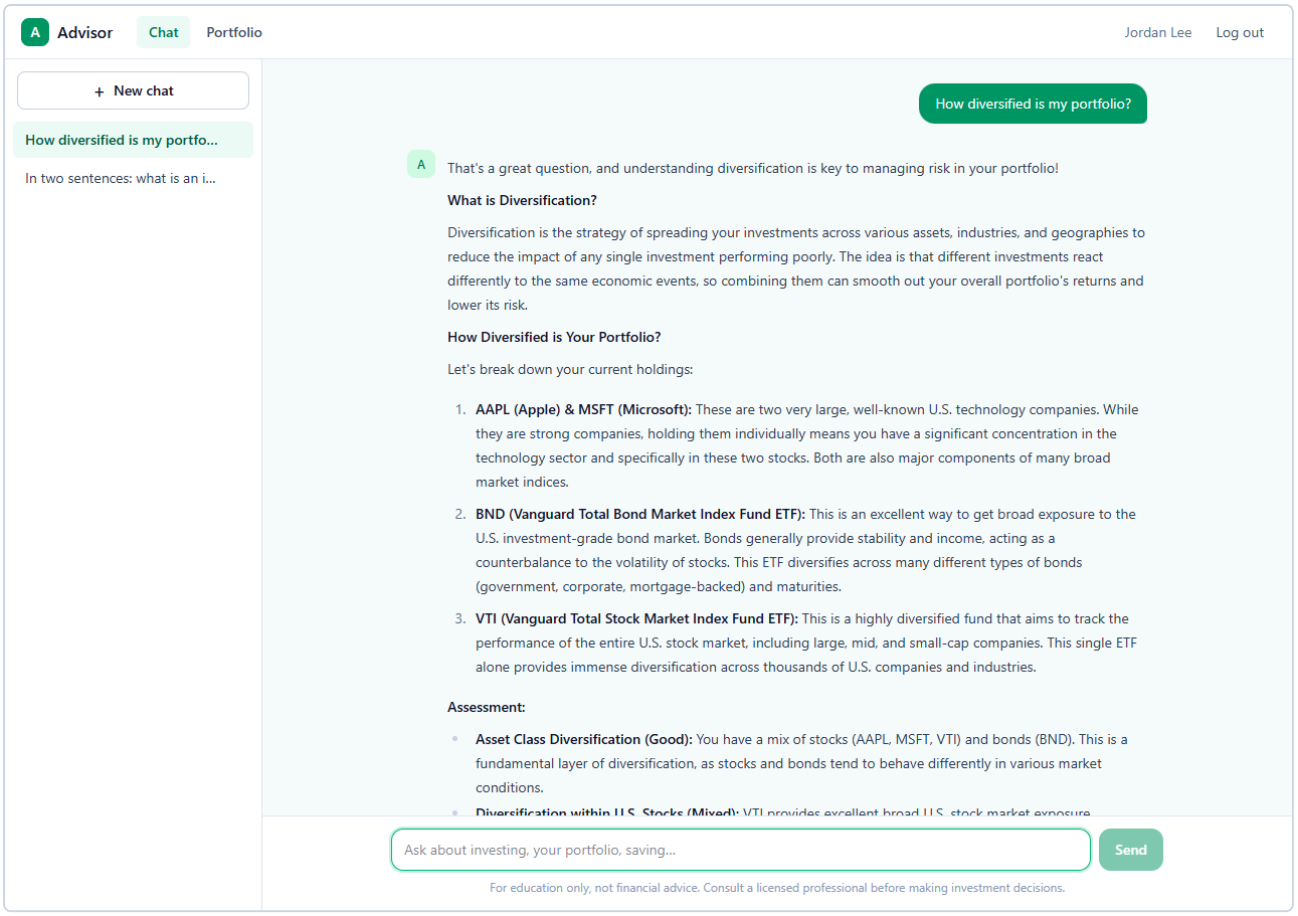


Empty chat with suggested questions

While the advisor works you'll see **Advisor is thinking...**, and then the answer appears word by word.



Advisor is thinking



The advisor's answer, referring to the user's own holdings

The advisor remembers the earlier messages in the same conversation, so you can ask follow-ups like "what would make it more diversified?".

Good questions to try:

- How diversified is my portfolio?
- Explain index funds vs. individual stocks
- How much should I keep in an emergency fund?
- What is dollar-cost averaging?
- What are the risks of holding individual tech stocks?

If something goes wrong (for example the AI service is busy), a red message appears above the input box and your question is put back so you can send it again.

Manage your conversations

- Each chat is listed in the left sidebar, named after its first question. Click one to reopen it.
- **New chat** starts a fresh conversation.
- Hover over a chat in the sidebar and click **×** to delete it. There's no confirmation step, and the messages are removed permanently.

Add your holdings

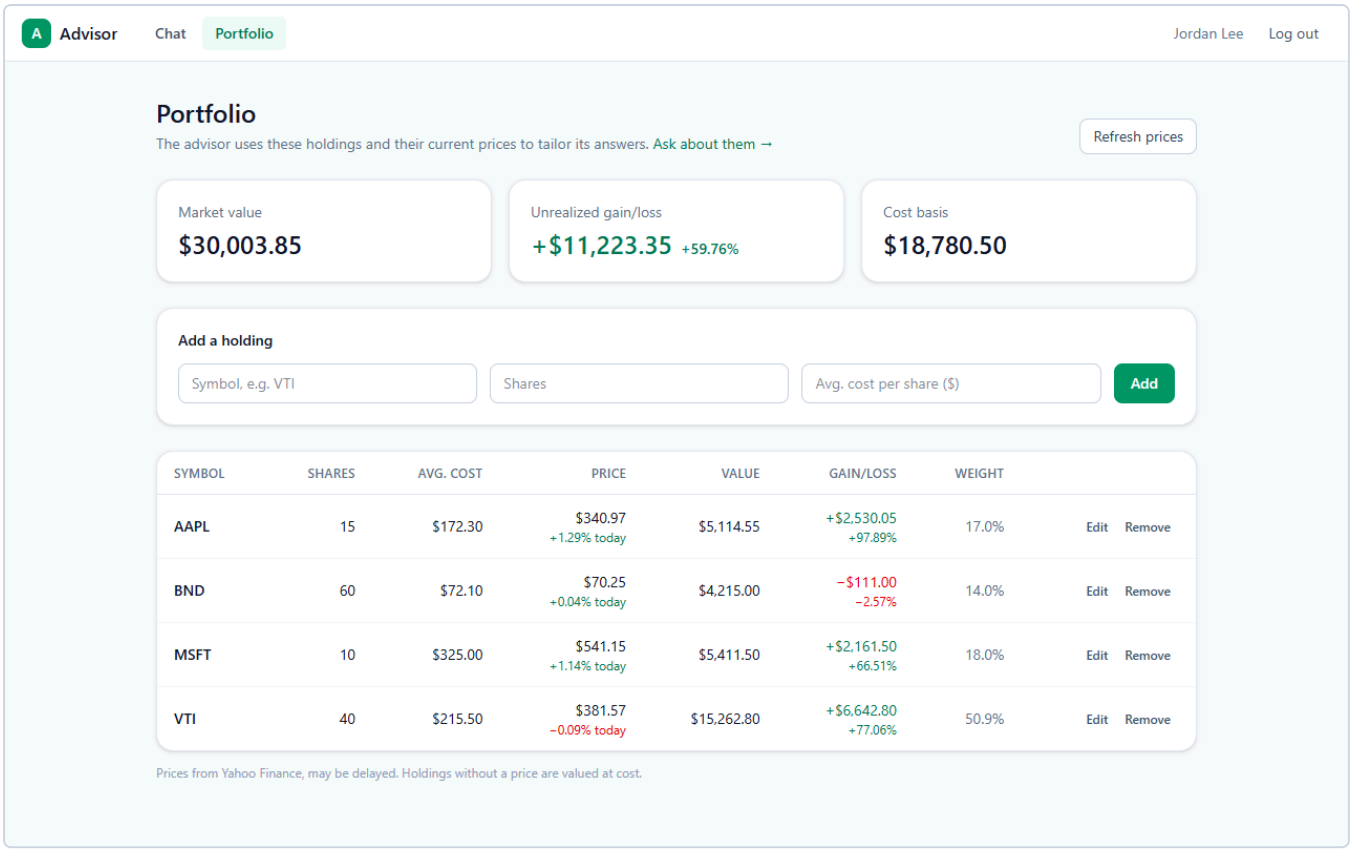
1. Click **Portfolio** in the top bar.
2. Under **Add a holding**, enter:

- **Symbol**: the ticker, e.g. `VTI`, `AAPL`, `BND`. Non-US tickers use Yahoo's suffix, e.g. `SHOP.TO`.
- **Shares**: how many you own (fractions are fine).
- **Avg. cost per share ($)**: what you paid per share on average.

3. Click **Add**.

Each symbol can appear only once. To change the amount, edit the existing row.

Read your portfolio



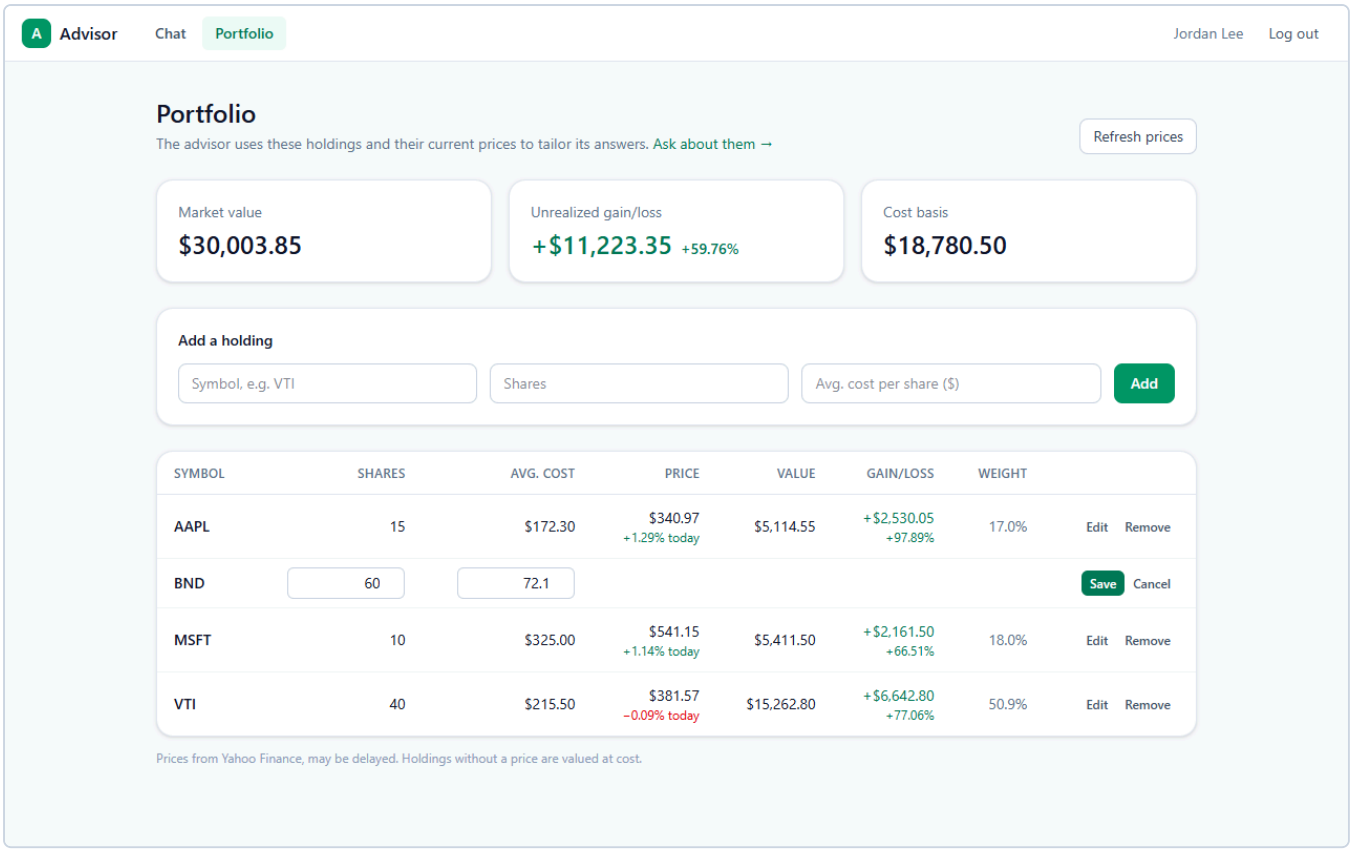
Portfolio page with live prices

Item	Meaning
Market value	What your holdings are worth at current prices
Unrealized gain/loss	Market value minus what you paid, in dollars and percent. "Unrealized" because you haven't sold
Cost basis	What you paid in total (shares × average cost)
Price	Latest price from Yahoo Finance, with today's change below it
Value	Shares × price
Gain/loss	Per holding, in dollars and percent
Weight	This holding's share of your total portfolio value

- Green means up, red means down.
- Prices may be delayed. Click **Refresh prices** to update them (they're re-fetched at most once a minute).
- If a price can't be found, the row shows **n/a** and that holding is counted at what you paid.

Edit or remove a holding

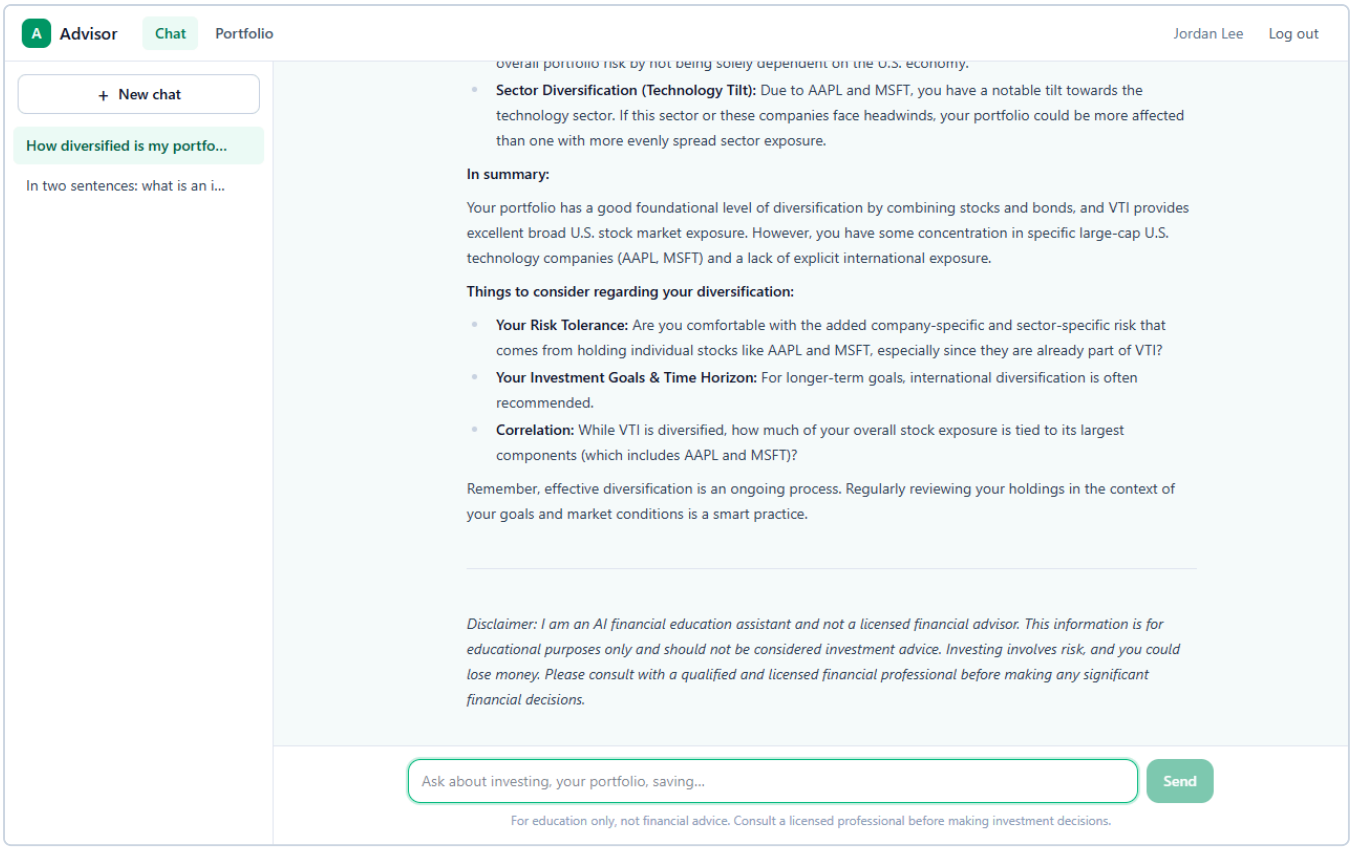
- **Edit**: change the shares or average cost, then **Save** (or **Cancel**).
- **Remove**: click it, then **Delete** to confirm or **Keep** to cancel.



Editing a holding

Understand the advisor's answers

- **It knows your portfolio.** Each answer is written with your current holdings and their live prices in view. In the example above it noticed the tech concentration from AAPL and MSFT, and that VTI already contains both.
- **It explains; it doesn't decide for you.** Expect explanations of concepts like diversification, risk, fees and time horizon, and questions to think about. It won't promise returns or tell you exactly what to buy.
- **It can be wrong.** AI models sometimes state things confidently that are out of date or incorrect. Check important facts, such as fund fees, with the fund provider.
- **Long answers can be cut off.** If an answer ends with "*(This answer was cut off because it hit the length limit.)*", ask it to continue or to be shorter.
- **For education only.** The note under the chat box is a reminder: this is not financial advice. The advisor often ends longer answers with a summary, points to consider and a disclaimer of its own:

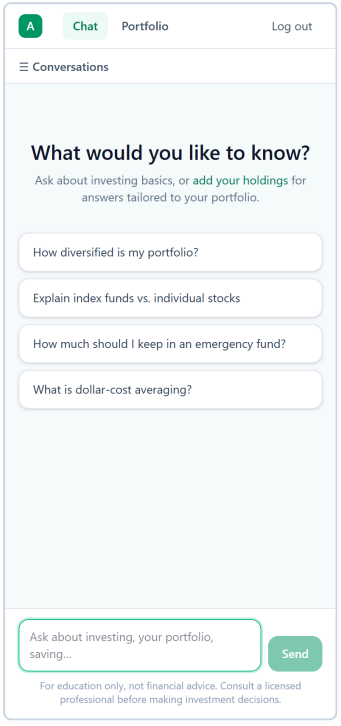


End of an answer: summary, things to consider and disclaimer

On your phone

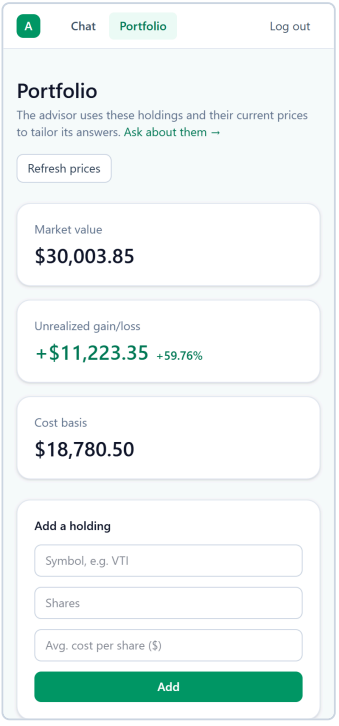
The app works on mobile browsers. On small screens the chat list moves behind ☰ **Conversations**.

Chat



Mobile chat

Portfolio

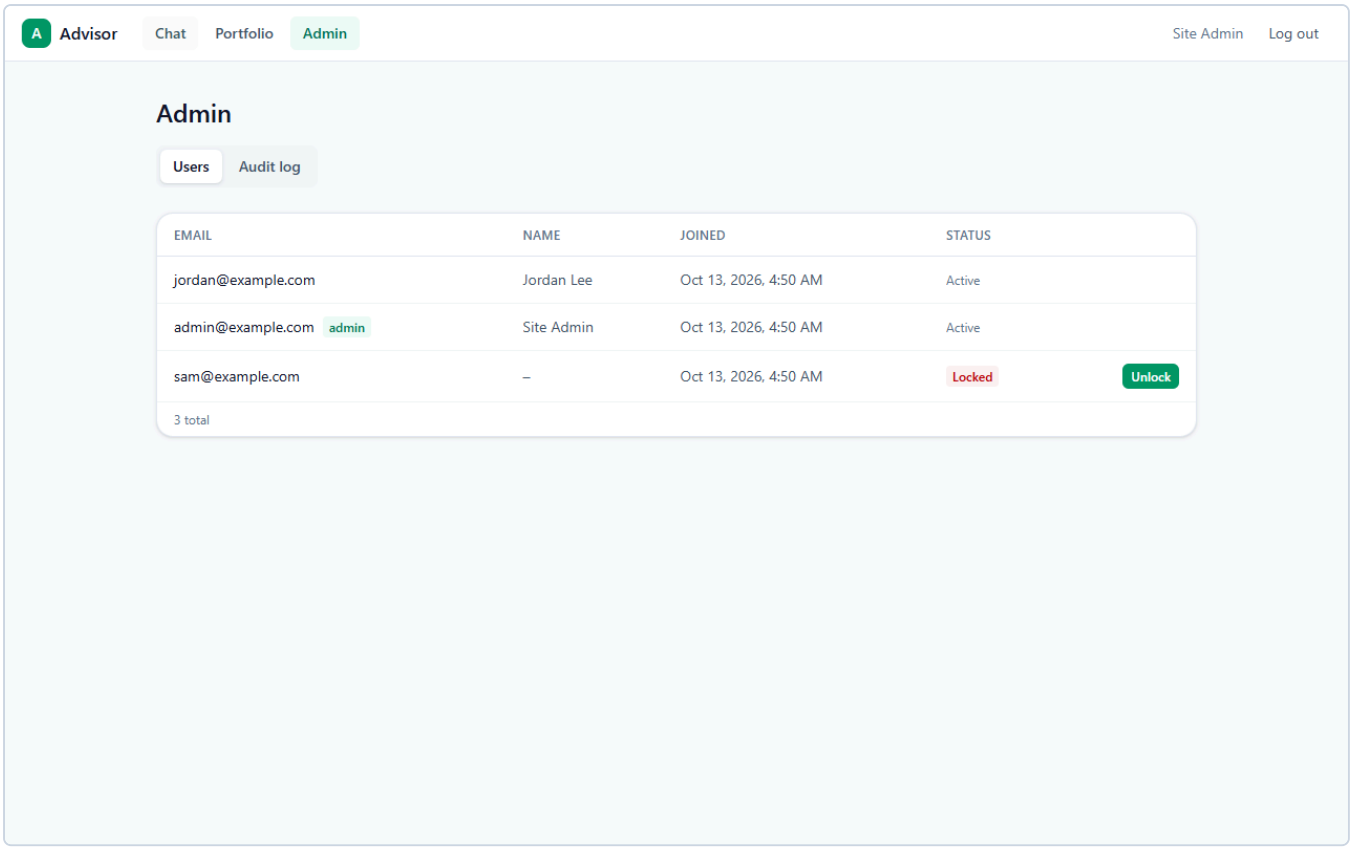


Mobile portfolio

Admin features

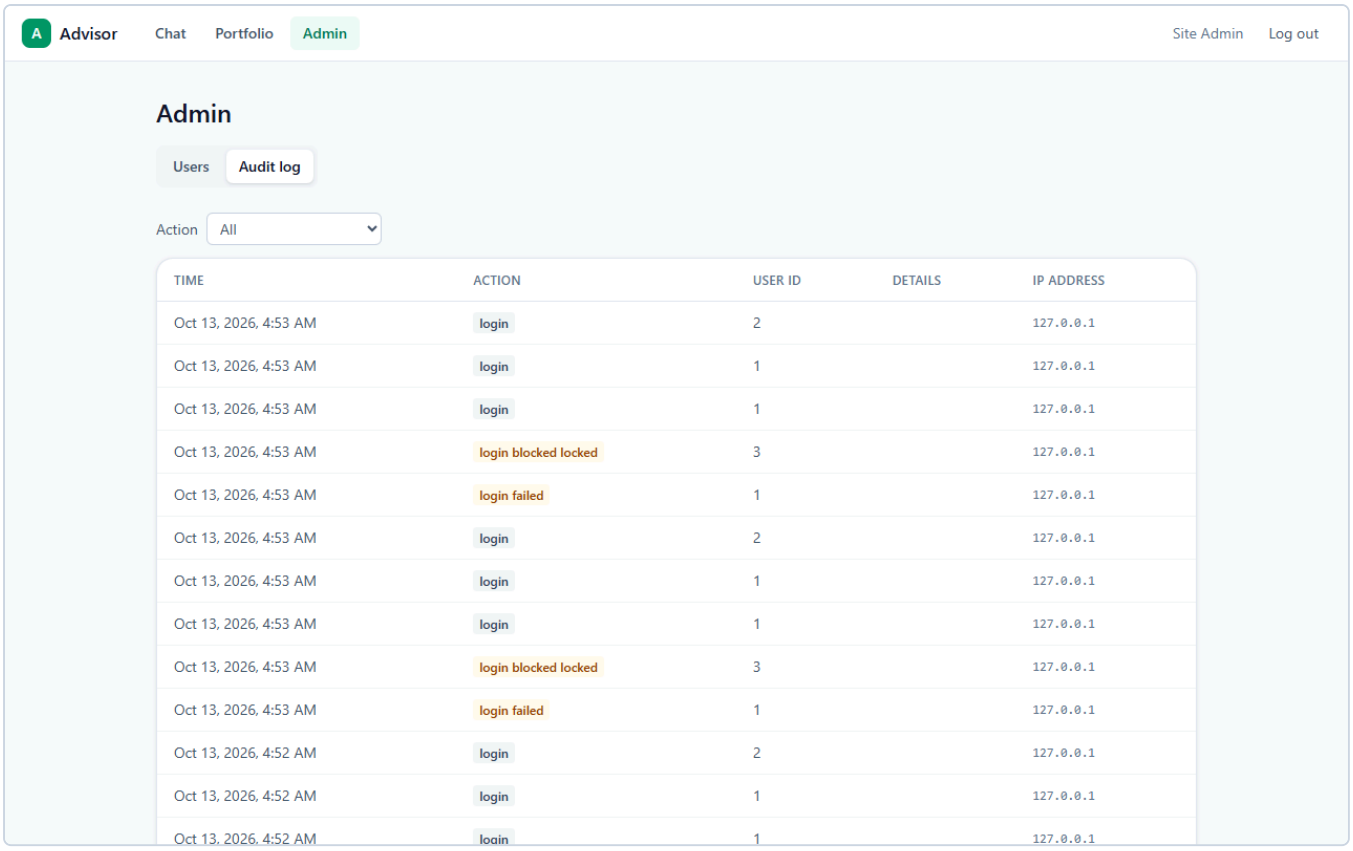
Administrators see an extra **Admin** tab.

Users lists every account with its join date and status. Locked accounts show a red **Locked** badge and an **Unlock** button; unlocking also resets the failed-password count.



Admin users tab

Audit log shows sign-ups, logins, failed logins, blocked logins, unlocks and portfolio changes, with the time, user ID and IP address. Use the **Action** dropdown to filter, e.g. to see only failed logins. Failed and blocked events are highlighted in amber.



Admin audit log

Admin rights are granted by whoever runs the server, with `flask --app main make-admin <email>`. Log out and back in afterwards to see the tab.

FAQ

Is this financial advice? No. It's an educational tool. Consult a licensed financial professional before making significant decisions.

Who can see my portfolio and chats? Other users can't see them. In the app, administrators see your email, name and account activity: logins, and which symbols you added, changed or removed. They don't see your chat messages, share counts or costs. Your messages and holdings are sent to Google's Gemini AI to generate answers, and stock symbols are sent to Yahoo Finance to get prices.

Do I need to enter my holdings? No. Without holdings you can still ask general questions; answers just won't be tailored to you.

Why is a price missing or different from my broker's? Prices come from Yahoo Finance and can be delayed. A missing price usually means the symbol is unknown to Yahoo; check the ticker.

I got "Too many requests". To keep the service fair there's a limit of 10 questions per minute and 200 per day. Wait a minute and try again.

I forgot my password. There's no self-service reset yet. Ask an administrator.

Can I delete my account? Not from the app yet. You can delete individual conversations and holdings; ask an administrator to remove the account.

Can I connect my brokerage account? No. Holdings are entered by hand, and the app never asks for brokerage logins.

See Also

- TROUBLESHOOTING.md if something isn't working